

Read Me!

Read Me~

Created by [HoneyCase](#), Jay Seb (credit due to using Jay's private team guides)

Welcome to the guide to Modding LifeGen. This is not a guide to adding or using mods with LifeGen - **this is for creating new content!**

Status: *Still a work in progress.*

(I am no longer actively available - it will take me longer to finish guides)

Please understand that LifeGen changes our features frequently, so this guide could become out of date at any time. If you ever have any questions, go to either #questions or #tech-help for answers!

Disclaimer: The LifeGen Developers are *far* from being professionals at coding. This game is purely a fun project and something to practice with, not a professionally thought-out game. There will be mistakes and spaghetti code.

Tips / Resources

If you're looking to publish your mod, consider using [github desktop](#) & its repository features. It's easier to navigate than doing it all purely through the terminal.

Do not mod on an install you actively play on. Backup your saves beforehand if you're doing something that potentially messes with your saves

I recommend using [visual studio code](#) for modding

- ★ Acts as both a code editor and terminal, "open folder" option, mass search and replace feature, merge & comparison features, able to create .json at your convenience.

Utilize ClanGen's [documentation](#) - Though not necessarily a guide, it's a great resource if you're looking for explanations of certain parts of the game.

Learn the basics of Python and Git. Modding is easier when you have a brief idea of what you're doing. Both are very beginner friendly

- ★ There are many great, free resources out there. Youtube, [python.org](#), [freecodecamp.org](#), [w3schools](#), etc etc. Obviously, you don't need to be an expert. Don't be afraid to search things up as you go.

Use the files available to you!

- ★ In the [lifegen github](#), you're able to search through the source files without actually downloading source. If you're simply looking to do a text mod, this can be quite helpful!
- ★ Most abbrevs are in utility.json / Possible tags are all listed in the features corresponding .py file or utility.py in scripts

[Pronoun Guide](#)

- ★ Clangen's guide to adding pronoun tags to your text. This is what makes the options he/she/they/etc available and used

[Pygame documentation](#)

- ★ If you want to take a step further in your modding beyond what is explained here, this is a helpful guide to see the ins and outs!

Table Of Contents

If you're looking for something specific and it's not explained here (tentative reminder that this document is mostly explaining LifeGen exclusive stuff), politely ask for the addition in #questions.

"Abbrevs" Tab: Lists LifeGen added abbrevs that you can use for lifegen specific content

"Feature: Dialogue" Tab: Explains how to create your own dialogue, plus adds all the necessary tags/abbreviations you'll potentially use.

"Feature: Elder's Stories" Tab: Explains how to create your own Stories for the Elder's Feature. This also adds necessary tags you'll potentially use.

(not started) "Feature: MC events" Tab: Explains what "MC events" are and how to create your own.

(not started) "Feature: MC patrols" Tab: Explains how to create your own Dark Forest, Date, and MC solo patrols. These are sort of similar to ClanGen's patrols, but they have their own things that make a guide necessary.

"Feature: Favourite Stars" Tab: Make your own addition star colors for favoriting cats

(unfinished) "Art Additions" Tab: Extensive and large tab explaining how to add to most art areas of the game.

Abbrevs

Possible Abbrevs

LifeGen uses many different shorthand abbreviations to refer to different types of cats in dialogue, events, and MC patrols.

NOTE: LifeGen added Abbrevs DO NOT WORK for ClanGen exclusive areas of the game. Thoughts, short events, ClanGen Patrols, etc CANNOT use these.

★ *Any LifeGen features that use ClanGen logic also do not use these abbrevs*

y_c = your cat's name

t_c = the name of the cat you're talking to

r_c = random cat

→ r_c1 = random cat 1

→ r_c2 = random cat 2

→ r_c3 = random cat 3

→ These are not compatible with just "r_c"

!! r_c cannot be used to grab a random cat in patrols, as, in patrols, r_c refers to the second cat in the patrol group.

!!! Please keep in mind that this numbering system **cannot** be used elsewhere unless specified in this doc. The numbers cannot be dynamically added, the numbered abbrevs are manually defined. Some mistakes I've seen in the past are attempts to use "r_k2" or "o_c_n2". Doesn't work like that!

c_n = clan name (previously c_nClan, now refers to full name)

o_c_n = other clan name (previously o_cClan, now refers to full name)

w_cClan = enemy clan if you're at war (w_c only refers to prefix)

l_n = leader's name

d_n = deputy's name

m_n = your mentor's name

tm_n = their mentor's name

`df_m_n` = your df mentor's name
`t_df_mn` = their df mentor's name

`y_a` = your apprentice's name
`t_a` = their apprentice's name

`r_k` = random kit
`r_a` = random app
`r_w` = random warrior
 → `r_w1` = random warrior 1
 → `r_w2` = random warrior 2
 → `r_w3` = random warrior 3
 → `r_w4` = random warrior 4
 → These are not compatible with just "r_w"

`r_m` = random med
`r_d` = random mediator
`r_q` = random queen
`r_e` = random elder
`r_s` = random sick cat
`r_i` = random injured cat

`y_m` = your mate
`y_k` = your kit
`y_kk` = your kit (kitten aged)
`y_p` = your parent
`y_s` = your sibling
`y_l` = your littermate

`t_m` = their mate
`t_k` = their kit
`t_kk` = their kit (kitten aged)
`t_ka` = their kit (adult aged)
`t_p` = their parent
 → **`t_p_negative`** = their parent- negative relationship
 → **`t_p_positive`** = their parent- positive relationship
`t_s` = their sibling
`t_l` = their littermate

n_r1 = Potential mate for n_r2 (adult, single, non-related)
n_r2 = Potential mate for n_r1 (adult, single, non-related)

yourcrush = your crush (romantic > 10, non-mates)
theircrush = t_c's crush (romantic > 10, non-mates)

tg_c = the cat t_c is grieving
yg_c = the cat you are grieving

d_c = a random dead cat of any residence
rsc_c = a random StarClan cat
rdf_c = a random Dark Forest cat
rur_c = a random UR cat

rsh_c = random shunned cat
rsh_k = random shunned kit
rsh_a = random shunned apprentice
rsh_w = random shunned warrior
rsh_m = random shunned medicine cat or medicine cat apprentice
rsh_d = random shunned mediator or mediator apprentice
rsh_q = random shunned queen or queen's apprentice
rsh_e = random shunned elder
sh_d = random shunned deputy
sh_l = random shunned leader
l_c = random lost cat
e_c = random exiled cat
r_g = random grieving cat

Abbrev Addons

{**relationship**}-{**abbrev**}-{**cluster/trait/skill**}

The **relationship** describes how t_c feels about the cat being referred to.

- plike (20)
- plove (50)
- rlike (10)
- rlove (50)

- dislike (20)
- hate (50)
- jealous (20)
- comfort (20)
- trust (20)
- respect (20)
- neutral (*everything under 20*)

The **cluster/trait/skill** refers to the cluster, trait or skill of the cat being referred to.

- This can be any cluster, trait, or skill, as listed in the guide (but all lowercase)
- You cannot have an abbrev with both a cluster/trait and a skill, as they have to go in the same place

Example uses:

- **hate-r_w** => A warrior that t_c hates.
- **r_c-strange** => A random strange cat.
- **your_crush-sweet** => A sweet cat you're crushing on.
- **plike-r_k-stable** => A random, stable kit t_c has 20+ platonic like for.
- **comfort-r_w-fighter** => A random warrior with the FIGHTER skill that t_c is comfortable with.

These work with pronouns!

E.g. {PRONOUN/plove-r_e-brooding/subject/CAP},
{VERB/plove-r_e-brooding/jump/jumps}

Note: In patrols and moon events, t_c will be replaced with you! So **plike-r_w** would be a warrior that your main character likes.

Feature: Dialogue

Dialogue

If you're ever confused, look at existing dialogue.

Welcome to LifeGen's biggest, most extensive feature ever: Dialogue. This is what makes LifeGen unique from ClanGen, and it takes up most of our time when writing for the game lol.

Disclaimer: LifeGen just recently reworked their dialogue, so it's still in the small test & fix phase. There will likely be changes as the developers figure out what works and what doesn't.

File + Dialogue Placements

Dialogue is placed in your game files. Navigate through your game folder > _internal > resources > dicts > lifegen_talk. There are quite a few JSONs in there, I'll explain their uses here:

Focuses

The Focuses folder is where we place all dialogue that's part of a "focus". This includes dialogue that only shows up during special dates.

- "Focus" dialogue is where for a period of time, cats will talk about a specific topic. These are triggered either by special dates or specific events.
- **All dialogue in these JSONs will have the "focus" or special date tags in "tags"**

NOTE: Focus JSONs are NOT specific in any way. If you intend for the dialogue to be only for kittens up, make sure you're constraining properly, otherwise newborns will get it

april_fools.json/new_years.json: Nothing is in here yet!

hailstorm.json: This is a focus given after a hail storm, which is triggered by an event. Cats will talk about a RECENT hail storm. This is post-hailstorm, not during 😊

halloween.json: Special date focus! Dialogue given during the halloween time frame will be placed here

leader.json: This is a focus given after a recent leader decision. Cats will talk about their approval of the leader, primarily based on their relationship with them

starving.json: This is a focus given when the clan is starving. Not triggered by an event, but instead by the state of the clan. Dialogue put here needs to have the starving condition in "condition": [] and "focus" in tags

unknown_murder.json: This is a focus triggered by an unrevealed murder (?). Cats discuss the murder, who they think did it, their feelings on the matter, etc

valentines.json: (not a special date specific thing) Triggered some time in newleaf, cats will periodically give each other flowers, decorate the camp, etc.

war.json: While the clan is currently at war, cats will discuss war in their dialogue. Not triggered by an event, but instead by the state of the clan.

Status JSONs

These JSONs dictate who the t_c is in the dialogue so it can be weighted properly and given to the correct cats. This goes by status (aka role) - some jsons are for y_c specifically.

NOTE: If your dialogue has multiple statuses for t_c, you'll need to paste the dialogue in each individual status JSON. Please make use of the exclusion status tags ("not_{status}")

apprentice.json: Place warrior apprentice t_c dialogue here

choice_dialogue.json: A JSON that weights the frequency of choice dialogue. No longer needed, really, but it exists

★ (aka you don't need to put choice dialogue here but if you want it weighted properly, you probably should make use of it)

crush.json: A JSON that weights the frequency of crush dialogue. Dialogue that mentions a crush (theircrush/yourcrush) will be placed here

Deputy.json: Place deputy t_c dialogue here

Elder.json: Place elder t_c dialogue here

Exiled.json: You are not able to speak to alive exiled cats - this file is a precaution if that situation is ever accidentally possible

Flirt.json: This JSON is for flirting dialogue. Make sure to add the tags "accept" or "reject" when adding flirting here

★ Adolescents can flirt with each other. Adults up can flirt with each other as long as they're within the 40 age moon gap rule

Former Clancat.json: Dead former clancat t_c dialogue goes here

General_no_kit.json: This is a catch all for when t_c has no status restrictions but kittens and newborns (for both t_c/y_c) are not intended to get the dialogue.

General_no_newborn.json: This is a catch all for when t_c has no status restrictions but newborns (for both t_c/y_c) are not intended to get the dialogue.

General_outsider.json: Dead outsider t_c (all kittypets, rogues, loners, etc can give this dialogue) dialogue goes here.

General_you_kit.json: This is for when t_c DOES NOT have a status but y_c is constrained for kitten or newborn.

- **NOTE:** If you put a dialogue where t_c is constrained here, it will ignore it

General.json: This file is a precaution if something ever completely goes wrong with dialogue

Insults.json: This is where you place insult dialogue. Make sure to add the tag "insult"

Kitten.json: Place kitten t_c dialogue here

Kittypet.json: Place dead kittypet t_c dialogue here

Leader.json: Place leader t_c dialogue here

Loner.json: Place dead loner t-c dialogue here

Mediator apprentice.json: Place mediator apprentice t_c dialogue here

Mediator.json: Place mediator t_c dialogue here

Medicine cat apprentice.json: Place medicine cat apprentice t_c dialogue here

Medicine cat.json: Place medicine cat t_c dialogue here

Newborn.json: Place newborn t_c dialogue here

Queen.json: Place queen t_c dialogue here

Queen's apprentice.json: Place queen's apprentice t_c dialogue here

Rogue.json: Place dead rogue t_c dialogue here

Warrior.json: Place warrior t_c dialogue here (this does not include leader and deputy)

LifeGen Compliant

If you wish to be compliant to how LifeGen does their dialogue, so you're consistent with the rest of the text, here are a few main rules we follow:

Brackets in text [] are used to describe actions. While text in quotes (") is verbal speaking.

```
"[t_c reluctantly drops a mouth-full of herbs at your paws,
huffing defiantly as they turn away.]",
"[You pick up the herbs.]"
```

```
"Hey, y_c, teach me the sign for mousebrain again. I need to use
it on hate-r_c.",
"[You roll your eyes and sign as requested. t_c copies it a few
times before a wide grin appears on their face. They run off, and
you hear hate-r_c's screaming in the distance.]"
```

y_c at no point talks, unless the dialogue is a choice dialogue and the talking is done through choice options. The only cat who does the talking is either t_c or the second cat in a multi cat dialogue.

★ Some older dialogue in LifeGen has y_c talking through the brackets [], but it is not the norm for today's set of rules

One line of dialogue is never more than five sentences long. This is because dialogue can be cut off - while we do have the scrollbar now, its preferred dialogue is broken up correctly to improve readability.

★ Some older dialogue is far too long, and LifeGen badly needs to revise these pieces of dialogue. Again, this isn't the norm for today's rules.

Bad dialogue example:

```
"You know, I never thought bees could be so vicious! When I was
out with r_w earlier, we found this huge nest and - you know -
honey is a great cure for the cough, so we thought; Hey! We
```

should get some honey for r_m. Well, we were wrong! Now look at me, I feel like a mossball filled with pine sticks!"

How we'd break it up:

"You know, I never thought bees could be so vicious!",
"When I was out with r_w earlier, we found this huge nest and - you know - honey is a great cure for the cough, so we thought; Hey! We should get some honey for r_m.",
"Well, we were wrong! Now look at me, I feel like a mossball filled with pine sticks!"

LifeGen dialogue is not, at any point, overly brutal, graphically detailed, under the NSFW umbrella, or have immoral/controversial topics.

- ★ !! If you make and publicize dialogue of this nature, your mod will have the risk of getting taken down.
- ★ If you find existing dialogue with these topics, please use [#open-a-ticket](#) on the server with a screenshot and an explanation as to why it should not be in LifeGen. Thank you.

Template

You *do not* need to use every option here. If you're not using a specific thing, you can leave it blank [] or delete it from your dialogue

Make sure your ID is unique. The best way to do that is to format your dialogue in a specific way: "blindtodead_honey1" "[topic]_[user id][number]"

- To use connected dialogue, add ~1,~2,~3 to the end of your id, like "Blindtodead_honey1~1". With every connected dialogue after that, go up a number

Full

```
"dialogue_id": {
  "biome": [],
  "season": [],
  "y_c": {
    "status": [],
    "age": [],
    "cluster": [],
    "condition": [],
    "skill": [],
    "dead": [],
    "backstory": [],
    "min_max_faith": [],
    "forgiven": false,
    "shunned": false
  },
  "t_c": {
    "status": [],
    "age": [],
    "cluster": [],
    "condition": [],
    "skill": [],
    "dead": [],
    "backstory": [],
    "min_max_faith": [],
    "forgiven": false,
    "shunned": false
  },
  "relationship": [],
  "tags": [],
  "intro": []
}
```

Choice Addition

If you want the dialogue to have choices, you'll add the below addition under "intro", like so:

```

...
"intro": [],
"intro_choices": {
  "choice1": {
    "text": "",
    "next_scene": "scene1"
  },
  "choice2": {
    "text": "",
    "next_scene": "scene2"
  }
},
"scene1": [],
"scene2": []
}
}

```

"text": The text that is displayed on the choice buttons. This can either be MC's "words" or actions. For example: "[Praise t_c.]" or "You did well."

"next_scene": is basically the ID of the dialogue that comes after choosing a specific option

- a single dialogue cannot have two of the same scene ID, otherwise they will pull from the same text. If that's not your intention, make sure the IDs are different
- You can leave it as "scene1", "scene2", etc - but make sure that if you do change the ID, the text that comes after "next_scene" is exact. For example: "next_scene": "poppy" should have a dialogue IDed as "poppy": []

Choice options can also have choices for the choices (goodness).

You'll follow the same as above, but instead of "intro", it'll be "scene1_choices".

Multiple Cat Dialogue

If you want the dialogue to use multiple cats, in the "intro", you'll start the dialogue line with `||` - inside the `||` has to be a valid abbrev, such as `r_c`, `r_w`, `t_l`, etc.

- The second cat *cannot* be `t_c` or `y_c`
- The second cat *cannot* say their own name
- The second cat (currently) is not constricted the same way `t_c` and `y_c` are. Conditions do not affect their dialogue. If second cat is blind and they mention seeing something, that can't currently be fixed

Example:

```
"intro": [  
    "Hello, y_c! Meet my new friend!",  
    "|d_c|Hi, it's nice to meet you!",  
    "[You can't help but scream. That's a ghost!]"  
]
```

There can be more than two cats! So if you want to make a dialogue that has 5 cats talking to each other, then that's completely possible.

Dialogue Affects

Dialogue can now affect specific things, such as giving accessories, relationship gains, and making cats join the df. (and likely more to come)

- ★ Scene effects will always be named "intro_scene_effects" if they're not a choice dialogue.
- ★ If they are, put in the ID for the scene you want to have an effect: "scene1_scene_effects".
 - More than one scene can have an effect but they must have their own section. You cannot do "scene1_scene2_scene_effects. It has to be separate

```
"intro_scene_effects": {  
    "inventory": {},
```

```

        "relationship": {},
        "dark_forest": {}
    }

```

"inventory"

```

    "intro_scene_effects": {
        "inventory": {
            "cats_to": ["t_c"],
            "accessory": ["DAISY"],
            "addition": "choice"
        }
    }

```

"cats_to": This is the abbrev of the cat(s) who gets the accessory.

"accessory": The list of potential accessories that can be given to the cat. These need to reflect the code names of accessories, not their display names. Use the [Visual Guide](#) for LifeGen

"addition": You have two options here - "choice" or "all".

- "choice" means the cat will get only one random accessory from the list,
- "all" means they'll get all the listed accessories at once.

"relationship"

```

    "intro_scene_effects": {
        "relationship": {
            "cats_to": ["y_c"],
            "cats_from": ["t_c"],
            "values": {
                "romantic": 0,
                "platonic": 0,
                "dislike": 0,
                "respect": 0,
                "comfort": 0,
                "jealousy": 0,
            }
        }
    }

```

```

        "trust": 0
    },
    "mutual": true
}

```

"cats_to": The cat(s) who is being felt differently about.

"cats_from": The cat whose feelings are being changed.

"values": The relationship values being changed. Can be positive or negative values. Make these smaller numbers (1 to 5, -5 to -1), since it's likely dialogue that is more or less "generic" will be given more often.

"mutual": Whether the feeling is mutual. Default is false.

"dark_forest"

- Avoid using abbrevs outside of t_c and y_c for this feature. It *can* work with other apprevs but it can also cause bugs - since random abbrevs are not able to constrain if the cat is a df trainee or not.
- Leave this for choice dialogue with the implied outcome, so the MC has a clear choice in the matter

```

"intro_scene_effects": {
    "dark_forest": {
        "join": ["t_c"],
        "leave": ["y_c"]
    }
}

```

"join": The abbrevs of the cats who are joining the Dark Forest.

"leave": The abbrevs of the cats who are leaving the Dark Forest.

Breakdown

"biome": []

The Clan must be in the listed biome(s) to get the dialogue.

Options:

- "mountainous"
- "plains"
- "forest"
- "beach"

"desert" and "wetlands" are not yet implemented by ClanGen, so they cannot be used

"season": []

The save must currently be in the listed season(s) to get the dialogue

Options:

- "newleaf"
- "greenleaf"
- "leaffall"
- "leafbare"

y_c/t_c blocks

These are the constraints for the talking cat (t_c) and MC aka your cat (y_c). These blocks are not for any other abbrev

"status": []

This constraints the status (aka role) of the cat. There can be more than one listed in any combination that is necessary for your dialogue.

Options:

- "newborn"
- "kitten"
- "apprentice"
- "mediator apprentice"
- "medicine cat apprentice"
- "queen's apprentice"

- "warrior"
- "leader"
- "deputy"
- "mediator"
- "medicine cat"
- "queen"
- "elder"

You can also use **exclusion tags**, which are just "not_" in front of the status options above. For example: "not_kitten" (also excludes newborns), "not_medicine cat apprentice", etc

- Do not replace the spaces with underscores

MISC tags:

- "df_trainee"
- "not_df_trainee"
- "guide" (the starclan/df guide)

Disclaimer: the role "young_elder" no longer exists as a constraint. Instead, constraint "status" with "elder" and "age" with "not_senior"

- Young elders are cats who have retired early due to a permanent condition

"age": []

The age(s) of y_c/t_c needs to be to get the dialogue

options:

- "newborn"
- "kitten"
- "adolescent"
- "young adult"
- "adult"
- "senior adult"
- "senior"

You can also use **exclusion tags**, which are just "not_" in front of the age options above. For example: "not_adolescent", "not_senior", etc

- Do not replace the spaces with underscores

"cluster": []

The cat has to have the cluster(s) and/or trait(s) to get this dialogue.

Your options can be found in the [Lifegen Text Guide](#)

- Since sweet is both a cluster and a trait, use "sweet" for cluster and "sweet_trait" for trait.

There are no exclusion tags available for clusters.

"condition": []

Indicating if t_c or y_c have a condition that is needed to get the dialogue.

options:

- Any illness, injury, or permanent condition. "heartbroken", "pregnant", "an infected wound", etc.
- "illness:any" and "injury:any" will work for generic injured/ill dialogue.
- "illness:none" and "injury:none" will completely exclude ill/injured cats, respectively. This includes all conditions, so it will also exclude cats that are grieving, pregnant, guilty, or heartbroken.
- "not:condition" will exclude cats with that condition from getting the dialogue. For example: "not:heartbroken"

You have **two options** on how to format the conditions (excluding perm conditions):

```
"t_c": {  
  "condition": [  
    "broken bone", "torn pelt"  
  ]  
},
```

^ For dialogue that requires one out of the listed conditions

```
"t_c": {
```



```
        "condition": [
            ["broken bone", "torn pelt"]
        ],
    },
```

^ For dialogue that requires ALL listed conditions

permanent condition specifications

Permanent conditions have some specialised tags. They follow this pattern:

"condition:born_with(true/false/any):exclusive(true/false)"

examples:

"deaf:true:true"

- The condition is **deaf**.
- The cat was **born with** this condition.
- This dialogue is **exclusive** to cats who were born deaf.

"blind:false:false"

- The condition is **blind**.
- The cat was **not born with** this condition.
- This dialogue is **not exclusive** to cats who went blind.
- This, however, means that cats who were born blind *cannot* get the dialogue as the first "false" still has its effect. This sort of tag would be rarely used.

"raspy lungs:any:true"

- The condition is **raspy lungs**.
- It **does not matter** whether or not the cat was born with the condition.
- This dialogue is **exclusive** to cats with raspy lungs.
- For permanent conditions that aren't blind or deaf, the exclusive value should always be true. The constraint will have no effect otherwise.

"hearing"

- The "hearing" tag is for a t_c who is explicitly mentioned to be able to hear in the text.

"skill": []

The cat has to have the skill(s) to get this dialogue

Skills can be found in the **† LIFE GEN SKILLS** . Skills are formatted like "skill": ["DARK,1"]

"[skill],[required skill tier]"

Disclaimer: If a cat has more than one skill in the constraint, they will only need one of the listed skills to get the dialogue.

"dead": []

If you want to constraint the dialogue to only be available for dead cats, use this block.

Options:

- "any"
- "df"
- "sc"
- "ur"

"alive" or "living" is not a valid tag. Do not put anything in the "dead" block if they're not dead

In addition, you can also control how many moons the cat has to be in the afterlife to get the dialogue

"min_deadfor_XX"

"max_deadfor_XX"

Examples:

"max_deadfor_10" > This dialogue is only possible when the cat has been dead **less** than ten moons.

"min_deadfor_100" > This dialogue is only possible when the cat has been dead **over** 100 moons.

"backstory: []"

The backstory the cat must have.

Options:

- "clanborn"
- "formerlyarogue"
- "formerlyaloner"
- "formerlyakittypet"
- "outsiderroots"
- "half-clan"
- "formerlyaoutsider"
- "orphaned"
- "abandoned"
- "ancientspirit"
- "originallyfromanotherclan"

You cannot constrict for a specific backstory, so "abandoned_3" will not work as a tag.

"min_max_faith": []"

Faith is a number. Positive numbers are StarClan while negative numbers are the Dark Forest. 0,0 is neutral.

Examples:

"min_max_faith": [-9,9] > any faith

"min_max_faith": [1,9] > only StarClan faith cats can get the dialogue

"min_max_faith": [-9,-1] > only Dark Forest faith cats can get the dialogue

"min_max_faith": [0,0] > only neutral cats can get this dialogue

Remember: it's negative to positive on the scale. Negative first, then positive. If it's all negative it'll be biggest before smallest (-8,-2)

Visual:

-9,-8,-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7,8,9

"forgiven"

The constraint that determines if the cat is forgiven or not forgiven

- "Forgiven" is the dialogue change after shunned (lasts 10 moons) so dialogue doesn't do a 180 from shunned to normal

If a dialogue is not tagged or using forgiven, it is defaulted to false

Options:

- "forgiven": true
- "forgiven": false

"shunned"

The constraint that determines if the cat is shunned, not shunned, or can get the dialogue regardless of shunned status

- "Shunned" is a condition that is given to a cat when they're revealed for murdering another cat. It affects text largely, and shunned cats can ONLY get shunned dialogue when they're shunned

Options:

- "shunned": true
- "shunned": false
- "shunned": "any"

"relationship: []"

These are relationship tags. They indicate what relationship t_c and y_c need to have *with each other* to get the dialogue

min/max relationship value tags

- *For example:* "min_platonic_XX", "max_respect_XX"
- **values:** platonic, romantic, dislike, respect, comfort, trust
- Relationship values range from 0-100. Each 10 points is one full bar on the cat's relationships screen

These values go off t_c's relationship to y_c. If it's a dead dialogue, it's flipped (since dead cats don't have relationships except mc)

Family

- "from_your_parent"
- "from_adopted_parent"
- "half_sibling"
- "littermate"
- "siblings_mate"
 - Akin to "sibling in law"
- "cousin"
- "adopted_sibling"
- "parents_siblings"
 - Akin to "aunt", "uncle", or unisex term equivalent
- "from_your_kit"
- "they_grandchild"
- "they_grandparent"
- "from_mate"
- "sibling"
- "from_adopted_kit"

Mentors or Apprentices

- "from_mentor"
- "from_df_mentor"
- "from_your_apprentice"
- "from_df_apprentice"

MISC

- "murderedthem"
- "murderedyou"
- "grievingthem"
- "grievingyou"
- "former_mate"

Exclusion tags

- "non-mates"
- "non-related"

NOTES: If you want a dialogue to be given for both blood related and adoptive, you need to put both tags. For example, "from_your_kit" and "from_adopted_kit" for a general MCs child dialogue

"tags": []

All the leftover stuff goes in the tags!

special dates

- "halloween"
- "new years"
- "april fools"

"Special dates" are content that are affected by real time events. If it's halloween for the player, halloween content will be given during that time, etc.

talk category tags

- "insult"
- "accept"
- "reject"

"accept" and "reject" tags are for flirting dialogue. This is what indicates if the dialogue is meant to be a reject or accept - remember, reject lower romantic like.

MC murder tags

- "murder_victim"
- "not_murder_victim"
- "accomplice"
- "accomplice_agreed"
- "accomplice_refused"
- "murder_success"
- "murder_fail"
- "murder_discovered"
- "murder_not_discovered"

misc

- "war"
- "clan_has_kits"
- "focus"
- "connected"
- "you_focuscat"
- "they_focuscat"
- "you_murderer"
- "you_not_murderer"
- "they_murderer"
- "they_not_murderer"
- "has_mate"

"focus" is a tag used for focus dialogue. Focus dialogue is when the clan talks about a specific thing for a handful of moons

- "focuscat" tags are for some focuses that talk about specific cats. "they_" is to indicate "t_c" is the focus, "you_" is to indicate "y_c" is the focus.

"connected" is a tag that handles the weight of connected dialogue. Put on connected dialogue only!

Feature: Elder's Stories

Elder Stories

If you are ever confused, look at existing stories

"Elder Stories" is a feature in LifeGen that involves Elder's telling tales to affect faith ("faith" is the belief system in LifeGen). These stories are written out and are able to be read by the player, plus they've sort of become a place to add stories based on what happened in the books.

★ (which is probably why people want to add to it)

File Placement

Elder stories can be found in: Game folder > _internal > resources > dicts > elder_stories.json

When adding your additions, make sure you're adding them under the appropriate section! The JSON is sectioned into "**starclan**", "**darkforest**", and "**neutral**".

"result_strings" can also be added to, but I think those are kinda straightforward in how they're made ^^;

LifeGen Compliant

If you wish to be compliant to how LifeGen does their stories, so you're consistent with the rest of the text, here are a few main rules we follow:

We have no rules about length. Stories can be as long or as short as you'd like them to be - just make sure to keep readability in mind while you're writing.

No dialogue in the story. Things such as "but wait didn't StarClan win in the end?" is not allowed

Template

```
{
  "id": "sc_J4",
  "min_max_cats": [1,1],
  "kits_amount": "none",
  "elder": {
    "min_max_faith": [-9, 9],
    "age": ["adolescent", "young adult", "adult"]
  },
  "random_cat": {
    "min_max_faith": [-9, 9],
    "status": ["apprentice", "warrior"],
    "cluster": ["assertive", "cool"]
  },
  "title": "Story Title",
  "story": [
    "Line 1.",
    "Line 2"
  ]
}
```

Breakdown

LifeGen abbrevs cannot be used, as text is processed with a ClanGen function. Only m_c, r_c, c_n, and o_c_n can be used.

- ★ m_c is the elder telling the story
- ★ r_c will be a random cat who is listening to the story.
- ★ c_n is the player clan name
- ★ o_c_n is a random other clan name

"id": A unique id to the story. Stories sharing ids will not work - one or the other will be unavailable to get.

"min_max_cats": How many cats the story needs to be given. **This can be upwards of 5**, but no further. For example, [3,5] requires there to be a minimum of 3 cats selected.

"kits_amount": Can be "none", "some" or "all". This dictates how many kittens need to be listening to the story for the text to be given.

"elder": {},

"elder": {} *is not necessary to have*. You can delete it completely if it doesn't have to be constrained, or just put "elder": {} as is

"min_max_faith": [-9, 9],

The faith constraints of the elder telling the story.

Examples:

"min_max_faith": [-9,9] > any faith

"min_max_faith": [1,9] > only StarClan faith cats can get the dialogue

"min_max_faith": [-9,-1] > only Dark Forest faith cats can get the dialogue

"min_max_faith": [0,0] > only neutral cats can get this dialogue

Remember: it's negative to positive on the scale. Negative first, then positive. If it's all negative it'll be biggest before smallest
(-8,-2)

Visual:

-9,-8,-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7,8,9

"age": ["adolescent", "young adult", "adult"]

This can be used to constrain if the elder is a young elder (a cat who retired early due to a permanent condition). Otherwise, you can delete it to be "age": []

"random_cat": {}

"random_cat": {} is not necessary to have. You can delete it completely if it doesn't have to be constrained, or just put "random_cat": {} as is

The random cat is a randomly chosen cat who is **listening** to the story.

"min_max_faith": [-9, 9],

The faith constraints of the elder telling the story.

Examples:

"min_max_faith": [-9,9] > any faith

"min_max_faith": [1,9] > only StarClan faith cats can get the dialogue

"min_max_faith": [-9,-1] > only Dark Forest faith cats can get the dialogue

"min_max_faith": [0,0] > only neutral cats can get this dialogue

Remember: it's negative to positive on the scale. Negative first, then positive. If it's all negative it'll be biggest before smallest (-8,-2)

Visual:

-9, -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

"age": ["adolescent", "young adult", "adult"]

This can be used to constrain r_c's age. If not necessary, "age": []

"status": ["apprentice", "warrior"],

This can be used to constrain r_c's status. If not necessary, "status": []

"cluster": ["neurotic"]

This can be used to constrain r_c's cluster (not trait)

"title": "": the title of your story. Keep it short, simple, but still relevant to the story's text.

"story": where you'll add your text. Like dialogue, this is line by line, usually broken up for readability. Avoid overly large paragraphs

- **NOTE**: Keep in mind that your story will always be followed by something in the results_strings dict. These can be found in the elder_stories.json file.

Elder Stories aren't supposed to be insanely specific, so there isn't a lot of constraining you'll have to do here!

Feature: MC Events

MC events

(aka events that are specifically only for MC)

DISCLAIMER: The creation of these events are *old as fuck*. At some point in the future, they will (hopefully) be revised!! But not anytime soon.

Many, MANY of the events that are going to be covered in this section ARE NOT EASILY CONSTRAINED. What does this mean? There is absolutely no support in most of these to be specific. You're given the bare minimum to work with.

For me

Game folder > resources > dicts > events > lifegen_events

Abbrevs from the tab above can be used in these events, except for murder.json and events that are meant for a specific constriction of cats

- ★ Affair.json: The event that is given after the player chooses to use the affair feature.
- ★ Ceremonies.json: The events that are given when MC is born, becomes an apprentice, becomes a full role, gains an app, gains a mate, has kits, when MC dies, etc
- ★ Df.json: events for when mc has joined the dark forest (?)
- ★ *Events_distrust.json: doesn't appear to be used anywhere*
- ★ Events.json: stuff like crush events, Mate events, and mc's mate cheating events are held
- ★ Gift.json: This is where the text for the gifting feature are held
- ★ Mate_lifegen.json: where the reject events are held for the confess pop up

- ★ Murder_punishments.json: the punishment flavor text events given when mc is shunned from the clan
- ★ Murder_unsuccessful.json: the events given when the mc is discovered for murder (murder feature)
- ★ Murder.json: the events given when mc used the murder feature and they succeed in killing the victim
 - Needs its own section
- ★ Nursery_activities.json: the text for the nursery activities
- ★ *Prophecies.json: unused*
- ★ Revival.json: the events given after mc revives in game

Shunned folder: events for shunned mc

Events folder: events for alive mc

Events w afterlives: events for dead mc

Moonplace: might have to be its own section - the events, prophecies, and dialogue for the moonplace

Feature: MC Patrols

"I serve you -- nothing!"

Feature: Favourite Stars

Favourite Stars

LifeGen adds more color options to the favorite cat feature, so now players can color code their cats - and even filter "all events" with said colors!

You will, however, need the source version. Since this involves editing the script files.

PNGs

Half of this work involves actually making your circles and stars. Since that's the easiest part of adding them, I'll go over that first.

- ★ You can totally use the files that're already there and just recolor them or use your art programs "hue" option

For **each** color option you want to add, you'll need the following pngs created ({num} refers to number. So _{num} would be _1 while file naming):

Added to resources > images:

- ★ fav_marker_{num}

Added to resources > images > buttons:

- ★ fav_star_{num}
- ★ fav_star_{num}_hover
- ★ fav_star_{num}_unavailable
- ★ fave_filter_{num}
- ★ fave_filter_{num}_hover
- ★ fave_filter_{num}_selected
- ★ fave_filter_{num}_selected_hover
- ★ fave_filter_{num}_unavailable

Since LifeGen already provides 3 colors, **your colors should start with 4 up!** Unless, of course, you plan on replacing the colors LifeGen provides with something else.

Once those are created, add them to a *separate* folder so you can keep the files if something ever goes wrong. You can then copy and paste your art to the LifeGen files once you're happy with them.

JSON files

Since the favoriting file components are buttons (except for fav_marker), we'll have to add their name/path to the game files to indicate that they're actually clickable. We'll be editing 5 files:

- ★ {resources > theme} buttons_small.json
- ★ {resources > theme} buttons.json
- ★ {resources > theme} image_buttons.json
- ★ {resources > theme} master_screen_scale.json
- ★ {resources > theme > generated} screen_scale.json

Thankfully it's pretty simple, since all we're going to do is copy what is already there and change the number in the name. I'll show how to do that for an example that adds a 4th option to the game below.

buttons_small.json

Search "#fav_star_3" in the file. This will take you to the fav star button code. All we'll do here is copy everything "#fav_star_3": {}, enter after the }, paste, then change the number to be 4:

```
},  
"#fav_star_3": {  
  "prototype": "#image_button",  
  "images": {  
    "normal_image": {  
      "path": "resources/images/buttons/fav_star_3.png"  
    },  
    "hovered_image": {  
      "path": "resources/images/buttons/fav_star_3_hover.png"  
    },  
    "selected_image": {
```

```

        "path": "resources/images/buttons/fav_star_3_hover.png"
    },
    "disabled_image": {
        "path":
"resources/images/buttons/fav_star_3_unavailable.png"
    }
}
},

```

Copy everything in blue as highlighted above:

```

    }
}
},
|
"#yourcat_filter": {
    "prototype": "#image_button",
    "images": {

```

Enter to make a space between }, and "#yourcat_filter"

```

        "disabled_image": {
            "path":
"resources/images/buttons/fav_star_3_unavailable.png"
        }
    }
},
"#fav_star_3": {
    "prototype": "#image_button",
    "images": {
        "normal_image": {
            "path": "resources/images/buttons/fav_star_3.png"
        },
        "hovered_image": {
            "path": "resources/images/buttons/fav_star_3_hover.png"
        },
        "selected_image": {
            "path": "resources/images/buttons/fav_star_3_hover.png"
        },
        "disabled_image": {
            "path":
"resources/images/buttons/fav_star_3_unavailable.png"

```

```

    },
    "#yourcat_filter": {
      "prototype": "#image_button",
      "images": {

```

The highlighted is what was pasted into the file. If yours doesn't look like this, please tweak it to do so.

Now, in what you pasted, you're going to change every 3 to be 4. Like so:

```

"#fav_star_4": {
  "prototype": "#image_button",
  "images": {
    "normal_image": {
      "path": "resources/images/buttons/fav_star_4.png"
    },
    "hovered_image": {
      "path": "resources/images/buttons/fav_star_4_hover.png"
    },
    "selected_image": {
      "path": "resources/images/buttons/fav_star_4_hover.png"
    },
    "disabled_image": {
      "path":
"resources/images/buttons/fav_star_4_unavailable.png"
    }
  }
},

```

Boom! You're done. You'll do this for EVERY color you added - just keep increasing the numbers (4, 5, 6, 7, etc) for each.

But we're not entirely done - this is just the example for fav_star. We'll also have to add "fave_filter_4" and "fave_filter_4_selected" to the file as well. ^^ It takes the same logic we applied here.

Here's an example.

```
"#fave_filter_4": {
  "prototype": "#image_button",
  "images": {
    "normal_image": {
      "path": "resources/images/buttons/fave_filter_4.png"
    },
    "hovered_image": {
      "path": "resources/images/buttons/fave_filter_4_hover.png"
    },
    "selected_image": {
      "path": "resources/images/buttons/fave_filter_4_hover.png"
    },
    "disabled_image": {
      "path":
"resources/images/buttons/fave_filter_4_unavailable.png"
    }
  }
},
"#fave_filter_4_selected": {
  "prototype": "#image_button",
  "images": {
    "normal_image": {
      "path":
"resources/images/buttons/fave_filter_4_selected.png"
    },
    "hovered_image": {
      "path":
"resources/images/buttons/fave_filter_4_selected_hover.png"
    },
    "selected_image": {
      "path":
"resources/images/buttons/fave_filter_4_selected_hover.png"
    },
    "disabled_image": {
      "path":
"resources/images/buttons/fave_filter_4_selected_hover.png"
    }
  }
},
```


From here, you can paste your additions to the button_small json to all the other jsons that were listed at the start of "JSON files". The code is the same across all the files (*idk why its like this dont ask me*)

Coding

Here's your reminder that adding stars involves having the source version.

Adding stars involves editing quite a few files, so bare with me.

Disclaimer: I haven't actually checked if these steps work out correctly, so if you follow this and get an error, I'm so sorry 😭 ask the tech help channel and I'll hopefully get to you

ui_elements.py

```
{scripts > game_structure > ui_elements.py}
```

Once you're in the file, search "# FAVOURITE ICON" (without quotes). This will take you to where we indicate how many star options are available. In my example, I'll only be adding one additional number

```
# FAVOURITE ICON
if show_fav:
    # LG: redone for multiple fav groups
    for num in [1, 2, 3, 4]:
        fav_indexes = [display_cats.index(cat) for cat in
display_cats if cat.favourite == num]
        [self.create_favor_indicator(i, self.bboxes[i], num) for i
in fav_indexes]
    # ---
```

Highlighted in blue is where I added to the file

If you want 6 options at the end of this, it'll be [1, 2, 3, 4, 5, 6], for example.

This is the only change you will make to this file.

ProfileScreen.py

```
{scripts > screens}
```

Once you're in the file, search "favorite_button" (with quotes). We'll change two minor things here.

```
elif event.ui_element ==
self.profile_elements["favourite_button"]:
    if self.the_cat.favourite == 4:
        self.the_cat.favourite = 0
    else:
        self.the_cat.favourite += 1
        self.clear_profile()
        self.build_profile()
else:
    self.handle_tab_events(event)
```

Change the 3 to be the highest color number you added (if you're adding 3 additional colors, put 6, etc)

```
self.profile_elements["favourite_button"] = UIImageButton(
    favorite_button_rect,
    "",
    object_id=fav_star_id,
    manager=MANAGER,
    tool_tip_text=f"Move to {self.the_cat.favourite + 1}" if
self.the_cat.favourite < 4 else "Remove favourite"
    if self.the_cat.favourite != 0
    else "Mark as favorite",
    starting_height=2,
    anchors={
        "right": "right",
        "right_target": self.profile_elements["cat_name"],
    },
)
```

Change the 3 to be the highest color number you added (if you're adding 3 additional colors, put 6, etc)

EventsScreen.py

```
{scripts > screens}
```

The edits we'll do to this file is gonna kinda be extensive, but rather simple since we already did half the work in previous steps.

Once in the file, mass search "fave_group_3". Anywhere this is mentioned will probably have some sort of edit applied to it.

We will first edit `all_filters = []` to have all the groups we intend to add. If you are adding an additional 3 stars, then you should have up to 6 groups.

```
self.all_filters = [
    "yourcat_filter",
    "fave_group_1",
    "fave_group_2",
    "fave_group_3",
    "yourcat_filter_selected",
    "fave_group_1_selected",
    "fave_group_2_selected",
    "fave_group_3_selected"
]

self.selected_filters = ["yourcat_filter_selected",
"fave_group_1_selected", "fave_group_2_selected", "fave_group_3_selected"]
self.unselected_filters = ["yourcat_filter", "fave_group_1",
"fave_group_2", "fave_group_3"]
```

Example on just adding a fourth option. You will need to do this for every number/group added (4, 5, 6, etc)

```
self.all_filters = [
    "yourcat_filter",
    "fave_group_1",
    "fave_group_2",
    "fave_group_3",
```

```

        "fave_group_4",
        "yourcat_filter_selected",
        "fave_group_1_selected",
        "fave_group_2_selected",
        "fave_group_3_selected",
        "fave_group_4_selected"
    ]
    self.selected_filters = ["yourcat_filter_selected",
        "fave_group_1_selected", "fave_group_2_selected", "fave_group_3_selected",
        "fave_group_4_selected"]
    self.unselected_filters = ["yourcat_filter", "fave_group_1",
        "fave_group_2", "fave_group_3", "fave_group_4"]

```

The highlighted blue is what I added to the file

We'll now edit this part of the code. Copy the code shown below. Like we did the button JSON files, we're simply going to copy, paste, and change the number

```

        elif (
            "fave_group_3" in self.fave_filter_elements and
            element == self.fave_filter_elements["fave_group_3"]
        ):
            self.fave_filter_elements["fave_group_3"].hide()
            self.fave_filter_elements["fave_group_3_selected"].show()
            self.selected_fave_filter.append("fave_group_3")
            self.place_fave_filters()
            self.handle_tab_switch("all events")
        elif (
            "fave_group_3_selected" in self.fave_filter_elements and
            element ==
self.fave_filter_elements["fave_group_3_selected"]
        ):
            self.fave_filter_elements["fave_group_3"].show()
            self.fave_filter_elements["fave_group_3_selected"].hide()
            self.selected_fave_filter.remove("fave_group_3")
            self.place_fave_filters()
            self.handle_tab_switch("all events")

```

Like so:

```
        elif (
            "fave_group_3" in self.fave_filter_elements and
            element == self.fave_filter_elements["fave_group_3"]
        ):
            self.fave_filter_elements["fave_group_3"].hide()
            self.fave_filter_elements["fave_group_3_selected"].show()
            self.selected_fave_filter.append("fave_group_3")
            self.place_fave_filters()
            self.handle_tab_switch("all events")

        elif (
            "fave_group_3_selected" in self.fave_filter_elements and
            element ==
self.fave_filter_elements["fave_group_3_selected"]
        ):
            self.fave_filter_elements["fave_group_3"].show()
            self.fave_filter_elements["fave_group_3_selected"].hide()
            self.selected_fave_filter.remove("fave_group_3")
            self.place_fave_filters()
            self.handle_tab_switch("all events")

    elif (
        "fave_group_4" in self.fave_filter_elements and
        element == self.fave_filter_elements["fave_group_4"]
    ):
        self.fave_filter_elements["fave_group_4"].hide()
        self.fave_filter_elements["fave_group_4_selected"].show()
        self.selected_fave_filter.append("fave_group_4")
        self.place_fave_filters()
        self.handle_tab_switch("all events")

    elif (
        "fave_group_4_selected" in self.fave_filter_elements and
        element ==
self.fave_filter_elements["fave_group_4_selected"]
    ):
        self.fave_filter_elements["fave_group_4"].show()
        self.fave_filter_elements["fave_group_4_selected"].hide()
        self.selected_fave_filter.remove("fave_group_4")
        self.place_fave_filters()
        self.handle_tab_switch("all events")
```

Everything in blue is what is added to the file.

Do this for every group number added

Next part~ Navigate to this part and copy all of this

```
self.fave_filter_elements["fave_group_3"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3",
    manager=MANAGER,
    container=self.event_screen_container
)
```

Enter after the bracket

```
self.fave_filter_elements["fave_group_3"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3",
    manager=MANAGER,
    container=self.event_screen_container
)
|
self.fave_filter_elements["yourcat_filter_selected"] =
UIImageButton(
```

Paste and change the numbers as shown below. Repeat for every group added

```
self.fave_filter_elements["fave_group_4"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 4 - 5), (25,
height))),
    "",
    object_id="#fave_filter_4",
    manager=MANAGER,
    container=self.event_screen_container
)
self.fave_filter_elements["yourcat_filter_selected"] =
UIImageButton(
```

(I do not know what happens if you go over 5 in the height so um. Experiment if you want more than 5 color star options on the event filtering tab)

Scroll a little until you see this code

```
self.fave_filter_elements["fave_group_3_selected"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3_selected",
    manager=MANAGER,
    container=self.event_screen_container
)
```

Do the same you did above - enter after bracket) and copy & paste the code for group_3_selected. Repeat for every color group added

Like so:

```
self.fave_filter_elements["fave_group_3_selected"] =
UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3_selected",
    manager=MANAGER,
    container=self.event_screen_container
)

self.fave_filter_elements["fave_group_4_selected"] =
UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 4 - 5), (25,
height))),
    "",
    object_id="#fave_filter_4_selected",
    manager=MANAGER,
    container=self.event_screen_container
)
```

Navigate to this code ("fave_group_3"), enter after the bracket), copy & paste code then change the numbers

```
self.fave_filter_elements["fave_group_3"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3",
    manager=MANAGER,
    container=self.event_screen_container
)
self.fave_filter_elements["fave_group_4"] = UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 4 - 5), (25,
height))),
    "",
    object_id="#fave_filter_4",
    manager=MANAGER,
    container=self.event_screen_container
)
```

Do the same here

```
self.fave_filter_elements["fave_group_3_selected"] =
UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 3 - 5), (25,
height))),
    "",
    object_id="#fave_filter_3_selected",
    manager=MANAGER,
    container=self.event_screen_container
)
self.fave_filter_elements["fave_group_4_selected"] =
UIImageButton(
    ui_scale(pygame.Rect((30, 287 + height * 4 - 5), (25,
height))),
    "",
    object_id="#fave_filter_4_selected",
    manager=MANAGER,
    container=self.event_screen_container
)
```


Navigate this code

```
if "fave_group_3" not in self.selected_fave_filter:
    self.fave_filter_elements["fave_group_3"].show()
    self.fave_filter_elements["fave_group_3_selected"].hide()
else:
    self.fave_filter_elements["fave_group_3"].hide()
    self.fave_filter_elements["fave_group_3_selected"].show()
```

We'll apply the same logic we did everywhere else. Enter twice after (), then paste the code above there. Change the numbers

```
if "fave_group_3" not in self.selected_fave_filter:
    self.fave_filter_elements["fave_group_3"].show()
    self.fave_filter_elements["fave_group_3_selected"].hide()
else:
    self.fave_filter_elements["fave_group_3"].hide()
    self.fave_filter_elements["fave_group_3_selected"].show()

if "fave_group_4" not in self.selected_fave_filter:
    self.fave_filter_elements["fave_group_4"].show()
    self.fave_filter_elements["fave_group_4_selected"].hide()
else:
    self.fave_filter_elements["fave_group_4"].hide()
    self.fave_filter_elements["fave_group_4_selected"].show()
```

HOPEFULLY that covers everything you need to do. Save everything if you haven't already and run the game. Crossing my fingers it works as intended and nothing goes amiss.

Art Additions

Art Additions

This was originally made to be a ClanGen Guide, so this does not have lifegen specific stuff currently. Still a work in progress - sorry for any one who got excited for the stuff I haven't done yet lol

Table of Contents

This guide does not go over high-res or additional sprites

[Art Additions](#)

[Table of Contents](#)

[Lists](#)

[List](#)

[Preparations](#)

[Layout](#)

[... If the layout file isn't big enough](#)

[What to Download](#)

[File placement](#)

[List of PNG names \(A-Z\):](#)

[Adding New Pelt Options](#)

[PNG](#)

[Coding](#)

[Sprites.py](#)

[Pelts.py](#)

[Adding New Color Options](#)

[PNG](#)

[Expanding Canvases](#)

[Coding](#)

[Sprites.py](#)

[Pelts.py](#)

[Colour_categories](#)

[Adding New Eye Colors](#)

[Expanding Canvases](#)

[Coding](#)

[Sprites.py](#)

[Pelts.py](#)

[Accessories](#)

[: Adding to existing PNG](#)

[PNG](#)

[Coding](#)

[Sprites.py](#)

[medcatherbs_data](#)

[Pelts.py](#)

[Cats.py & scar_events.py](#)

[acc_display.json](#)

[: Adding new PNG](#)

[PNG](#)

[Coding](#)

[Sprites.py](#)

[Pelts.py](#)

[Cats.py & scar_events.py](#)

[Utility.py](#)

[acc_display.json](#)

[Adding to White Patches](#)

[PNG](#)

[Coding](#)

[Sprites.py](#)

[pelts.py](#)

[Vit and Colorpoints](#)

[Adding to Tortie Pattern](#)

[PNG](#)

[Coding](#)

[sprites.py](#)

[pelts.py](#)

[Adding to Skins](#)

[Expanding Canvases](#)

[Coding](#)

[sprites.py](#)

[pelts.py](#)

[Adding to Scars](#)

[sprites.py](#)

[pelts.py](#)
[Adding to tints](#)
[Adding Symbols](#)
[Adding Camps](#)
[Adding Patrol Art](#)

Lists

You might see a lot of brackets (`[ ]`) in this guide. These are called [lists](#), and are exactly what the name suggests. They allow the coder to store multiple objects in a single variant, primarily for data storing purposes.

In clangen, you will see these a lot while coding sprite options. Lists allows clangen to specify what each sprite is named, where they are in the spritesheet, and what categories the sprites are in. Here's an example:

```
accessory_data = [  
    ["ONE", "TWO", "THREE"],  
    ["FOUR", "FIVE", "SIX"],  
],
```

It's easier to think of these as boxes. Everyone has seen those plastic dividers you put in your kitchen cabinets, right? Well, this functions the same way.

- "accessory_data" is the cabinet space holding all the objects. `[ ]` signifies the whole space.
- The brackets inside "accessory_data" are the dividers. They're placed inside the cabinet (`[ ]`) and they store their own objects (`[["ONE"]]`). Think of every row on the spritesheet as a divider

List

List of what to go over: (ALL OF THIS FOR STABLE)

☒ Preparations

- ☒ ~~Layout~~
- ☒ ~~What to download~~
- ☒ ~~File placement~~
- ☒ ~~Provide a drive with the boxes layout and lineart. Use the biggest file clangen has which is white patches~~
- ☒ ~~Adding pelt types~~
- ☒ ~~adding new color options~~
- ☒ ~~Adding new eye colors (for both eyes/eyes2)~~
- ☒ ~~Accessories (adding to existing file and adding new png)~~
- ☒ ~~Adding to white patches~~
- ☒ ~~Adding to tortie markings~~
- ☐ Adding to skins
- ☐ Adding to scars
- ☐ Adding tints (mention new tints would be more involved with the lifegen customizer)
- ☐ Adding symbols.... Ugh
- ☐ Adding a new camp

Preparations

Personally, I favor having everything laid out and planned before diving in. It gives me a sense of security and lays out a clear direction to follow. Otherwise, I just feel lost - and you kinda don't want that while modding.

If you're not that type of person or you get bored with planning, please refer to the [provided drive link](#)! This will have all the PNG files you may need starting out (**click the three dots, select details for connected descriptions**), but I still recommend at least reading through "layout"!

Layout

The PNG files have to be in a specific layout. The sprites are coded kinda like layers that are cut out and placed on the cats. If they're not laid out specifically, then you might experience pelts, eyes, etc "flying" off the cats.

- ★ If you want to know more about which order the sprites are layered or just how clangen does their sprites, please read [clangen's sprite documentation](#)!

Each sprite (or pose) is in a 50x50 pixel box. The sprites are 3x7 or 150x350, and there are a total of 21 poses. Each sprite option in the PNG is within the 150x350 barrier - if your art strays out of it, that's when you get stuff appearing in other sprite options. If that makes sense!

HOWEVER, there is a great way to avoid this! All it involves is the ~background colors~. Now, you may be wondering what the hell I mean. Check out [this drive](#) and click on the checker box png! This png is a "visual guide" of sorts. Example below.



Example of the whitepatches.png with my method

I use this method with practically every PNG within clangen's sprite folder. Personally, I see things better when the background is blue, so that's why mine is that way. The red lines are the "barrier", which show where the box stops and the new one starts.

And there you go! You should be safe from stray pixels, flying sprite options, or options intersecting into each other.

... If the layout file isn't big enough

If for some reason the provided file isn't big enough either in height or width, you need to do some math to extend the canvas size. Since each box is a 50 pixel box, the math should be really simple.

Here's how I do it with FireAlpaca (shout out to the easiest drawing program I ever used)

- Navigate to the top and select "edit"
- Select "canvas size..." (or ctrl + alt + c)
- Specify left if you need extend width, specify top center if you need to extend height
- IF extending width, add 150 x how many new rows you need
- IF extending height, add 350 x how many new rows you need

From there, copy the layer with the checker boxes and drag it over to align with the extended space. Merge the checker box layers, then do the same for the lineart layer if applicable.

What to Download

This kinda depends on what you want to do with the files, but it's generally good to note that **installing source gives you access to the script folder**. This is where all the backend code is held.

Want to add a new option? Want to rename a specific color? Want to add physical characteristics? Source.

Want to just do a retexture mod? Want to do a high-res mod? Want to do a rainbow mod that simply replaces the files that already exist? Application.

However, I highly recommend the source download for any type of modding. Yes, it's difficult to install compared to application, but having access to the code is always a win in my books. Of course, though, it's not necessarily needed if you're not adding anything new.

Outside of the clangen version, you also need:

1. A program that offers pixel brushes (or anti-aliasing). It doesn't have to be fancy.
2. Access to the sprite files for ClanGen.
 - If you're a phone user, you can install all the files individually via [Github](#) (view the file you want to download, select the three dots at the top, and select download)
3. A lot of time and patience depending the complexity of your mod

Optional, but I provide a [drive linking](#) to png files that generally make it easier to add or edit sprites! Please look at "layout" for more information.

File placement

If you've never looked in the sprite folder before, you might get a bit overwhelmed. That's okay! I'll list what's in the folder here and group them by function/section.

List of PNG names (A-Z):

- Remember that ClanGen developers are a mixed bag of regions and languages, so make sure you're naming your files exactly to the original! Just because one section of the game has american spelling, doesn't mean all of them will

Pelts: (14)

agouticolours
bengalcolours
classiccolours
mackerelcolours
marbledcolours
maskedcolours
rosettecolours
singlecolours
singlestripecolours
smokecolours
sokokecolours
speckledcolours
tabbycolours
tickedcolours

Linearts: (6)

aprilfoolslineart
aprilfoolslineartdead
aprilfoolslineartdf
lineart
lineartdead
lineartdf

Accessories: (6)

bellcollars
bowcollars
collars
medcatherbs
nyloncollars
wild

Fading: (3)

fadedarkforest
fademask
fadestarclan

Light and Shadows: (2)

lightingnew
shadersnewwhite

MISC: (9)

error_placeholder
eyes
eyes2
missingscars
scars
skin
symbols
tortiepatchesmasks
whitepatches

Faded Folder:

The Faded folder contains the fading sprites you usually see when a cat is completely faded. These are individual pngs of a single pose for each age group depicting either SC or DF

faded_adol
faded_adol_df
faded_adult
faded_adult_df
faded_kitten
faded_kitten_df
faded_newborn
faded_newborn_df
faded_senior
faded_senior_df

Dicts Folder:

Where the tints are held! These are not actual pngs - rather JSONs that hold code specifying the tint colors and which pelt color gets them naturally

```
tint.json
white_patches_tint.json
```

Paralyzed Folder:

***NOTE:** This Folder does not actually have a function anymore, but I believe the ClanGen developers keep it in the sprites folder to credit the original artist*

Adding New Pelt Options

Be aware you need the source (raw code) version of clangen to add additional options

So you want to add a new pelt option! A brief rundown: Pelt options are pelt patterns such as singlecolour, singlestripe, bengal, ect. They come in an assortment of different colors and typically range in complexity!

In this section, I'm going to use "spotscolours" as an example!

PNG

First, you'll need to actually draw your new pelt option. Refer to layout if you haven't already!

For uniformity, name your png like "[peltname]colours".

Now, the order of the color options within the file does matter! They're categorized separately in the code, so putting them in the png out of order will simply incorrectly label your colors. Below is the order of colors - each [] is a row in the png:

```
color_categories = [
    ["WHITE", "PALEGREY", "SILVER", "GREY", "DARKGREY", "GHOST",
"BLACK"],
    ["CREAM", "PALEGINGER", "GOLDEN", "GINGER", "DARKGINGER",
"SIENNA"],
```

```
["LIGHTBROWN", "LILAC", "BROWN", "GOLDEN-BROWN", "DARKBROWN",  
"CHOCOLATE"],  
]
```

Found on line 370 in sprites.py

Now before you go ham, I recommend using a method called "masking". This method involves making a separate layer for each detail section of the pelt. This makes it so much easier to create color options for pelts, as all you need to do is duplicate the folder, drag and align the new folder to the next option, and recolor each new layer.

Here's a finicky example of a potential mask for spotscolour:



Obviously color in the lines lol

Every color you see here is its own layer within a single folder. I usually call these "templates". I personally like to go light to dark so I understand my process while creating the mask, and this gives you the opportunity to plan out how you want your pelt(s) to look, so to speak.

You might not see the vision because of my eye bleeding art skills but I promise masking the pelts like this will help so much in the long run, especially if you decide you want to add more colors later.

Make sure to delete the lineart and background layer when you're done!

Otherwise it might look a little strange in game. Just a tad.

Coding

Once the PNG is created and you're happy with how it looks, you're going to plop it directly into the sprites folder. Make sure to write the name of the PNG somewhere as you're going to need it later while coding it in.

Sprites.py

{scripts > cat folder}

First file we're going to edit is sprites.py. This can be found within the path: scripts > cats > sprites.py. Scroll down to about like 150.

The first thing you have to do when adding a new png into the game is to make sure the game knows it's there. We do that by editing "for x in []"

```
for x in [  
    "lineart",  
    "lineartdf",  
    "lineartdead",  
    "eyes",  
    "eyes2",  
    "skin",  
    "scars",  
    "missingscars",  
    "medcatherbs",  
    "wild",  
    "collars",  
    "bellcollars",  
    "bowcollars",  
    "nyloncollars",  
    "singlecolours",  
    "speckledcolours",  
    "tabbycolours",
```

```
        "bengalcolours",
        "marbledcolours",
        "rosettecolours",
        "smokecolours",
        "tickedcolours",
        "mackerelcolours",
        "classiccolours",
        "sokokecolours",
        "agouticolours",
        "singlestripecolours",
        "maskedcolours",
        "shadersnewwhite",
        "lightingnew",
        "whitepatches",
        "tortiepatchesmasks",
        "fademask",
        "fadestarclan",
        "fadeddarkforest",
        "symbols",
        "spotscolours",
    ]:
```

The highlighted part at the bottom is the png I want to add to the list. Do not add .png to the end! Just the name. Make sure it's surrounded by quotes and have an ending comma - "spotscolours",

From there, scroll through the file until you see color_types at around line 376. We're also going to add the new png to this list, like below.

```
color_types = [
    "singlecolours",
    "tabbycolours",
    "marbledcolours",
    "rosettecolours",
    "smokecolours",
    "tickedcolours",
    "speckledcolours",
    "bengalcolours",
    "mackerelcolours",
```

```
        "classiccolours",
        "sokokecolours",
        "agouticolours",
        "singlestripecolours",
        "maskedcolours",
        "spotscolours",
    ]
```

All this does is essentially lets the game know that the png you added is a pelt option.

Save, and we're done with this file!

Very easy, right?

Pelts.py

```
{scripts > cat folder}
```

Okay now navigate to the pelts.py, which is in the same place as your sprites.py. You don't need to scroll here, as you're met with sprite_names straight away.

What sprite_names essentially does is gives your pelt option a name! If I were to add spotscolours, I'd probably give them the name spots. Like below:

```
sprite_names = {
    "SingleColour": 'single',
    'TwoColour': 'single',
    'Tabby': 'tabby',
    'Marbled': 'marbled',
    'Rosette': 'rosette',
    'Smoke': 'smoke',
    'Ticked': 'ticked',
    'Speckled': 'speckled',
    'Bengal': 'bengal',
    'Mackerel': 'mackerel',
    'Classic': 'classic',
```



```

    'Sokoke': 'sokoke',
    'Agouti': 'agouti',
    'Singlestripe': 'singlestripe',
    'Masked': 'masked',
    'Spots': 'spots',
    'Tortie': None,
    'Calico': None,
}

```

You might be wondering - what's the point of putting this twice? Great question! It's to separate the difference between what is a pelt name and what can be a pelt base. As you can see, tortie and calico are pelt names but they can't exactly be a base (like you can't make a calico cat double calico).

Make sure to capitalize the left but don't capitalize the right!

```
'Spots': 'spots',
```

Scroll down until about line 54 where you see `tortiebases = []`. We're simply going to add the name of the pelt, like we did above

```

tortiebases = ['single', 'tabby', 'bengal', 'marbled', 'ticked',
'smoke', 'rosette', 'speckled', 'mackerel',
               'classic', 'sokoke', 'agouti', 'singlestripe',
'masked', 'spots']

```

Don't put a comma after your added pelt!

Scroll until about line 102. This is where we're going to specify the categories of pelts. There's five options: **tabbies**, **spotted**, **plain**, **exotic**, and **torties**. In my example, I'm putting my pelt into spotted but add yours to wherever fits the most.

```

tabbies = ["Tabby", "Ticked", "Mackerel", "Classic", "Sokoke",
"Agouti"]
spotted = ["Speckled", "Rosette", "Spots"]
plain = ["SingleColour", "TwoColour", "Smoke", "Singlestripe"]
exotic = ["Bengal", "Marbled", "Masked"]
torties = ["Tortie", "Calico"]

```

```
pelt_categories = [tabbies, spotted, plain, exotic, torties]
```

Make sure it's capitalized here!

Scroll until around line 990. This is the description of the pelt used in the alliances. You can be as detailed as you want, but make sure c_n is somewhere in the text. **c_n is referring to color_name!**

```
pattern_des = {  
    "Tabby": "c_n tabby",  
    "Speckled": "speckled c_n",  
    "Bengal": "unusually dappled c_n",  
    "Marbled": "c_n tabby",  
    "Ticked": "c_n ticked",  
    "Smoke": "c_n smoke",  
    "Mackerel": "c_n tabby",  
    "Classic": "c_n tabby",  
    "Agouti": "c_n tabby",  
    "Singlestripe": "dorsal-striped c_n",  
    "Rosette": "unusually spotted c_n",  
    "Sokoke": "c_n tabby",  
    "Masked": "masked c_n tabby",  
    "Spots": "spotted c_n"  
}
```

If you want to be true to how the pelts are described, make sure to only use at most two descriptive terms.

I honestly wasn't too sure about the rest of the description code, so I'm not going to cover that since it's not going to break your game if you don't edit it.

Adding New Color Options

Be aware you need the source (raw code) version of clangen to add additional options

Adding a new color option involves editing all the existing pelt pngs in your sprites folder, plus any additional pelt options you've added

if applicable, since colors are categorized in the code separately from the PNGs.

- ★ If you're unsure which files are pelt pngs, please refer to "file placement"

I recommend duplicating the pelt pngs, adding them to a separate folder, and renaming them to be "bengal modified" or similar just to distinguish them from the original files. Later you can rename them and replace the original with your own.

PNG

Now, obviously it can be kinda hard to follow the original pelts if there isn't an exact mask available. Some pngs have the mask for all sprites within the file, while others do not. You can choose to use the provided markings or make your own!

- ★ If you have access to the discord server, please look in [#sprites](#) pins for pattern templates made by doe6143

I recommend just drawing over the markings on the pelt in a bright, clearly distinguishable color (like hot pink, red, etc), then recoloring them to your heart's desire.



Example of this method while fixing sprites

NOTE: For uniformity, most of the pelt options are just markings placed upon the singlecolor pelt option.

If you want to be true to how it's originally done, start with singlecolor then slowly move to the more detailed pelts like bengal, using the singlecolor from before as the base underneath.

NOTE: Newborns are slightly different from the other sprites! This is because naturally irl, cats don't usually pop out as what they'll look like as an adult.

★ Obviously as a mod creator you don't have to follow this detail.

Make sure to delete the lineart and background layer when you're done! Otherwise it might look a little strange in game. Just a tad.

Expanding Canvases

If you're adding multiple different colors at once, then you will need to expand the canvas size to do what you want. Here's how I do it with FireAlpaca:

- Navigate to the top and select "edit"
- Select "canvas size..." (or ctrl + alt + c)
- Specify top center since we're extending height
- Add 350 x how many new rows you need
 - ◆ If you're planning to add 7-14 new colors, then it'll be 350 x 2, for example

If your program doesn't work the way mine does, please consider searching up "how to expand canvas side in [art program]" in your browser

Coding

Once your colors are done for every pelt file and you're happy with how they look, replace the files in the sprites folder with your modified one. Or just save if you're editing the files directly.

From here, **name your colors**. Write down the order in which the colors are placed within the pngs. For this section, I am going to use "fawn" and "rusty" as examples.

Sprites.py

First, we'll be navigating to sprites.py. This file is in your scripts > cats . We'll only need to do one thing here.

Scroll down until line 369, color_catagories. You're going to see a list of colors. Each bracket [] symbolizes one row.

```
color_categories = [  
    ["WHITE", "PALEGREY", "SILVER", "GREY", "DARKGREY", "GHOST",  
    "BLACK"],  
    ["CREAM", "PALEGINGER", "GOLDEN", "GINGER", "DARKGINGER",  
    "SIENNA"],
```

```

        ["LIGHTBROWN", "LILAC", "BROWN", "GOLDEN-BROWN", "DARKBROWN",
"CHOCOLATE"],
        ["FAWN", "RUSTY"],
    ]

```

Make sure the names are UPPERCASE

Always, always follow what you added to the png. If you have fawn and rusty, but rusty is *underneath* fawn on the png instead of next to it, then you have to add rusty to a new row rather than putting them in the same bracket.

Pelts.py

Now, navigate it to pelts.py which is in the same area as sprites.py.

Scroll until line 30, pelt_colours. You will see three sections here: pelt_colours, pelt_c_no_white, pelt_c_no_bw.

- ★ **Pelt_colours** lists all the colors that can be generated
- ★ **Pelt_c_no_white** lists all the colors that are NOT white, aka WHITE
- ★ **Pelt_c_no_bw** lists all the colors that are NOT black, aka GHOST and BLACK

```

pelt_colours = [
    'WHITE', 'PALEGREY', 'SILVER', 'GREY', 'DARKGREY', 'GHOST',
'BLACK', 'CREAM', 'PALEGINGER',
    'GOLDEN', 'GINGER', 'DARKGINGER', 'SIENNA', 'LIGHTBROWN', 'LILAC',
'BROWN', 'GOLDEN-BROWN', 'DARKBROWN',
    'CHOCOLATE', 'FAWN', 'RUSTY'
]

pelt_c_no_white = [
    'PALEGREY', 'SILVER', 'GREY', 'DARKGREY', 'GHOST', 'BLACK',
'CREAM', 'PALEGINGER',
    'GOLDEN', 'GINGER', 'DARKGINGER', 'SIENNA', 'LIGHTBROWN', 'LILAC',
'BROWN', 'GOLDEN-BROWN', 'DARKBROWN',
    'CHOCOLATE', 'FAWN', 'RUSTY'
]

pelt_c_no_bw = [
    'PALEGREY', 'SILVER', 'GREY', 'DARKGREY', 'CREAM', 'PALEGINGER',

```

```
    'GOLDEN', 'GINGER', 'DARKGINGER', 'SIENNA', 'LIGHTBROWN', 'LILAC',  
'BROWN', 'GOLDEN-BROWN', 'DARKBROWN',  
    'CHOCOLATE', 'FAWN', 'RUSTY'  
]
```

Note: The indents don't matter. They're for readability purposes

My examples, fawn and rusty do not need anything fancy. However, if you're adding colors that are practically white or black, keep in mind that clangen filters those.

Alright, now we're going to scroll until line 110 - `sprite_colors`. It's basically the same deal as above, but now we're going to add our colors by category

```
single_colours = [  
    'WHITE', 'PALEGREY', 'SILVER', 'GREY', 'DARKGREY', 'GHOST',  
'BLACK', 'CREAM', 'PALEGINGER',  
    'GOLDEN', 'GINGER', 'DARKGINGER', 'SIENNA', 'LIGHTBROWN', 'LILAC',  
'BROWN', 'GOLDEN-BROWN', 'DARKBROWN',  
    'CHOCOLATE', 'FAWN', 'RUSTY'  
]  
  
ginger_colours = ['CREAM', 'PALEGINGER', 'GOLDEN', 'GINGER',  
'DARKGINGER', 'SIENNA', 'RUSTY']  
  
black_colours = ['GREY', 'DARKGREY', 'GHOST', 'BLACK']  
  
white_colours = ['WHITE', 'PALEGREY', 'SILVER']  
  
brown_colours = ['LIGHTBROWN', 'LILAC', 'BROWN', 'GOLDEN-BROWN',  
'DARKBROWN', 'CHOCOLATE', 'FAWN']  
  
colour_categories = [ginger_colours, black_colours, white_colours,  
brown_colours]
```

And there you go! Save the file and test it out.

Colour_categories

If you decide you want to add a new category for colors, then follow this!

Okay, so! Let me first explain what the categories do. **It's a way to add weight (or give a chance) to each color group via inheritance (genetics)**. This makes sure the generation of colors match the clans population.

This could be useful if you want to add "special" colors that only generate once in a blue moon, or if you want to add colorful pelts (like green, purple, etc) but don't want them to overtake the clan.

First - name your category. As an example, I'm going to use "colourful_colours" for the duration of this section.

Navigate to line 110 (#SPRITE NAMES) within your pelts.py. We're going to add the category here.

```
single_colours = [
    'WHITE', 'PALEGREY', 'SILVER', 'GREY', 'DARKGREY', 'GHOST',
'BLACK', 'CREAM', 'PALEGINGER',
    'GOLDEN', 'GINGER', 'DARKGINGER', 'SIENNA', 'LIGHTBROWN', 'LILAC',
'BROWN', 'GOLDEN-BROWN', 'DARKBROWN',
    'CHOCOLATE', 'GREEN', 'RED', 'RAINBOW'
]
ginger_colours = ['CREAM', 'PALEGINGER', 'GOLDEN', 'GINGER',
'DARKGINGER', 'SIENNA']
black_colours = ['GREY', 'DARKGREY', 'GHOST', 'BLACK']
white_colours = ['WHITE', 'PALEGREY', 'SILVER']
brown_colours = ['LIGHTBROWN', 'LILAC', 'BROWN', 'GOLDEN-BROWN',
'DARKBROWN', 'CHOCOLATE']
colourful_colours = ['GREEN', 'RED', 'RAINBOW']
colour_categories = [ginger_colours, black_colours, white_colours,
brown_colours, colourful_colours]
```

Next, scroll til line 470 - #weights for each colour group

If you're confused - **this code indicates which color can be inherited by a parent.**

So using ginger_colours as an example, ginger-colored cats have the highest chance (40) to have gingered kittens and a slight chance (10) to have brown kittens. Black and white are not possible for ginger cats to have, unless parent 2 is a black/white cat.

```
if p_ in Pelt.ginger_colours:
    add_weight = (40, 0, 0, 10)
```

Add_weight goes in the order of : (ginger_colours, black_colours, white_colours, brown_colours)

With our example addition, colourful_colours, there will have to be a fifth section to the weights. We'll do that like this:

★ *With every additional category, you will need to add a new section (,0) to the weights and a new **elif p_ in** section to the file*

```
# Weights for each colour group. It goes: (ginger_colours,
black_colours, white_colours, brown_colours, colourful_colours)
```

```
weights = [0, 0, 0, 0, 0]
```

```
for p_ in par_peltcolours:
```

```
    if p_ in Pelt.ginger_colours:
```

```
        add_weight = (40, 0, 0, 10, 0)
```

```
    elif p_ in Pelt.black_colours:
```

```
        add_weight = (0, 40, 2, 5, 0)
```

```
    elif p_ in Pelt.white_colours:
```

```
        add_weight = (0, 5, 40, 0, 0)
```

```
    elif p_ in Pelt.brown_colours:
```

```
        add_weight = (10, 5, 0, 35, 0)
```

```
    elif p_ in Pelt.colourful_colours:
```

```
        add_weight = (20, 20, 0, 20, 40)
```

```
    elif p_ is None:
```

```
        add_weight = (40, 40, 40, 40, 40)
```

```
    else:
```

```
        add_weight = (0, 0, 0, 0, 0)
```

```
for x in range(0, len(weights)):
```

```
    weights[x] += add_weight[x]
```

```
# A quick check to make sure all the weights aren't 0
```

```
if all([x == 0 for x in weights]):  
    weights = [1, 1, 1, 1, 1]
```

All the highlighted blue is what I added to the file!

This should be all you need to do, unless you want to get fancy with alliance descriptions. I will not go over that right now, cause I simply just don't understand it lol - and it shouldn't break your game if you don't edit it.

However, if you want to look at it, mass search one of the categories in the file (like white_colours) and it should take you to the code.

Adding New Eye Colors

Be aware you need the source (raw code) version of clangen to add additional options

Eye colors are very simple - it's just 10 pixels max for each sprite. However, for eye colors you have to edit two files - eyes and eyes2. Eyes2 is the heterochromia file.

I recommend doing the eyes.png first, copy your additions to eyes2 and delete the left eye of each sprite.

Because of the simplicity of the file, I don't think I need to explain how to create eyes. Please check out "layout" for the png files I recommend using while making new options!

Expanding Canvases

If you're adding multiple different colors at once, then you will need to expand the canvas size to do what you want. Here's how I do it with FireAlpaca:

- Navigate to the top and select "edit"
- Select "canvas size..." (or ctrl + alt + c)
- Specify top center since we're extending height
- Add 350 x how many new rows you need

- ◆ If you're planning to add 7-14 new colors, then it'll be 350 x 2, for example

If your program doesn't work the way mine does, please consider searching up "how to expand canvas side in [art program]" in your browser

Coding

Once your eye colors are done and you're happy with how they look, replace the files in the sprites folder with your modified one. Or just save if you're editing the files directly.

Make sure to delete the lineart and background layer when you're done! Otherwise it might look a little strange in game. Just a tad.

Sprites.py

{scripts > cats} Scroll until line 183 - eye_colours

Here, we're going to add the eye colors you added to the png. I'll be using ORANGE as an example.

If it's in row two:

```
eye_colors = [  
    [  
        "YELLOW",  
        "AMBER",  
        "HAZEL",  
        "PALEGREEN",  
        "GREEN",  
        "BLUE",  
        "DARKBLUE",  
        "GREY",  
        "CYAN",  
        "EMERALD",  
        "HEATHERBLUE",  
        "SUNLITICE",  
    ],  
]
```

```
[
    "COPPER",
    "SAGE",
    "COBALT",
    "PALEBLUE",
    "BRONZE",
    "SILVER",
    "PALEYELLOW",
    "GOLD",
    "GREENYELLOW",
    "ORANGE",
],
]
```

If it's in a new row:

```
eye_colors = [
    [
        "YELLOW",
        "AMBER",
        "HAZEL",
        "PALEGREEN",
        "GREEN",
        "BLUE",
        "DARKBLUE",
        "GREY",
        "CYAN",
        "EMERALD",
        "HEATHERBLUE",
        "SUNLITICE",
    ],
    [
        "COPPER",
        "SAGE",
        "COBALT",
        "PALEBLUE",
        "BRONZE",
        "SILVER",
        "PALEYELLOW",
        "GOLD",
        "GREENYELLOW",
    ],
]
```

```

        ],
        [
            "ORANGE",
        ],
    ]

```

Always, always follow what you added to the png. It is very case sensitive!!

Pelts.py

{scripts > cats} Scroll to line 58 - eye_colours

We'll now be adding the eye color option to the pool, plus what color category it would fit into.

```

    eye_colours = ['YELLOW', 'AMBER', 'HAZEL', 'PALEGREEN', 'GREEN',
'BLUE', 'DARKBLUE', 'GREY', 'CYAN', 'EMERALD', 'PALEBLUE',
                'PALEYELLOW', 'GOLD', 'HEATHERBLUE', 'COPPER', 'SAGE',
'COBALT', 'SUNLITICE', 'GREENYELLOW', 'BRONZE', 'SILVER', 'ORANGE']
    yellow_eyes = ['YELLOW', 'AMBER', 'PALEYELLOW', 'GOLD', 'COPPER',
'GREENYELLOW', 'BRONZE', 'SILVER', 'ORANGE']
    blue_eyes = ['BLUE', 'DARKBLUE', 'CYAN', 'PALEBLUE', 'HEATHERBLUE',
'COBALT', 'SUNLITICE', 'GREY']
    green_eyes = ['PALEGREEN', 'GREEN', 'EMERALD', 'SAGE', 'HAZEL']

```

The indentation does not matter - it is for readability

Scroll to line 121 - eye_sprites

```

    eye_sprites = [
        'YELLOW', 'AMBER', 'HAZEL', 'PALEGREEN', 'GREEN', 'BLUE',
'DARKBLUE', 'BLUEYELLOW', 'BLUEGREEN',
        'GREY', 'CYAN', 'EMERALD', 'PALEBLUE', 'PALEYELLOW', 'GOLD',
'HEATHERBLUE', 'COPPER', 'SAGE', 'COBALT',
        'SUNLITICE', 'GREENYELLOW', 'BRONZE', 'SILVER', 'ORANGE'
    ]

```

Anddd you're done!

Accessories

Be aware you need the source (raw code) version of clangen to add additional options

Accessories are items the cats wear in game. Currently in stable, they can only wear one accessory at a time. This is changed in development

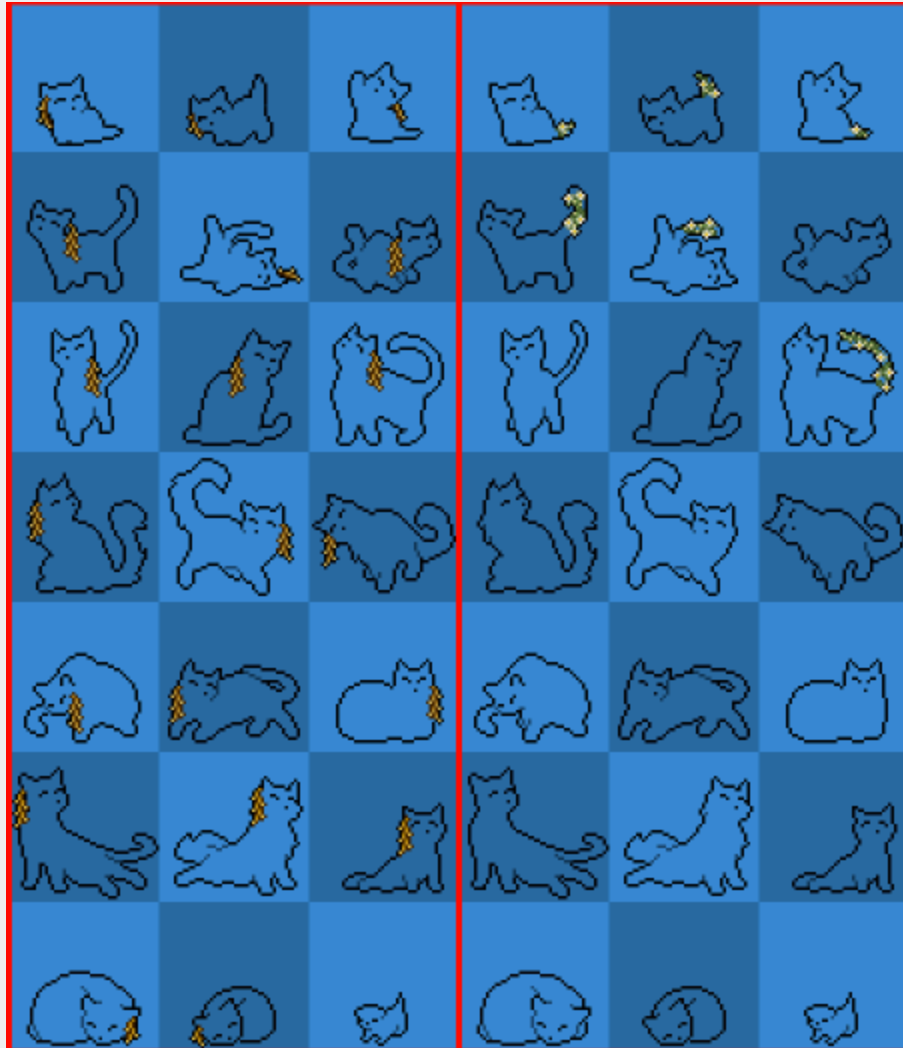
NOTE: Accessories cannot share names in the code. They can share display names, but you cannot have two "DAISY" accessories. This can be easily fixed by adding a number (DAISY2) or giving it a descriptive name (DAISYCLUSTER)

: Adding to existing PNG

PNG

This is the same process as adding a new color for pelts - all you need to do is simply add your additions to the existing PNG (in this example I'll be using medcatherbs) by using the layout png guides I provide in "layout".

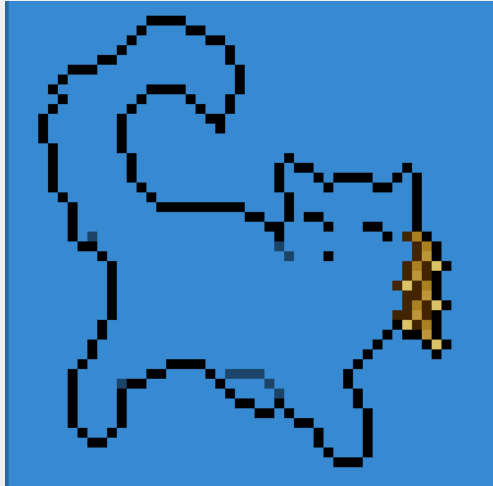
★ **NOTE:** Make sure when you're done, you're deleting the background checker box pattern and lineart layer. Otherwise your sprites will look a little weird. Just a tad.



Example adding an unfinished accessory, "arrowhead", to medcatherbs

NOTE: If you'd like to do it the same way ClanGen does, be aware that there are a few ground rules they follow.

- Accessories must cover less than 1/3rd of a cat
- The accessory must have a lineart. Inside the sprite is a darker color than the accessory (if it's a daisy, the lineart is a dark grey), while if it's outside the sprite, the lineart is black



- ClanGen uses shadow to dictate depth. If it's behind the cat, make it darker
- Be mindful of lost limb scars. Only no/half tail has code for certain accessories

Coding

Okay, once you're happy with your accessories, you can now save the file (if you're editing the png directly) or replace the png (if you duplicated the png to make edits).

In this section, I'm going to use the png "medcatherbs" and the accessory "ARROWHEAD" as examples.

Sprites.py

```
{scripts > cat folder}
```

Since we're adding to an existing file, we don't need to do anything fancy. We're just going to add our accessories to the PNGs code.

Search the file for the name of the png you edited. For me, it's going to be "medcatherbs". The first thing I'm going to edit is "medcatherbs_data". All you're going to do is add your accessory(ies) to the data, like below.

If you placed it beside an accessory:


```
medcatherbs_data = [  
    [  
        "MAPLE LEAF",  
        "HOLLY",  
        "BLUE BERRIES",  
        "FORGET ME NOTS",  
        "RYE STALK",  
        "CATTAIL",  
        "POPPY",  
        "ORANGE POPPY",  
        "CYAN POPPY",  
        "WHITE POPPY",  
        "PINK POPPY",  
    ],  
    [  
        "BLUEBELLS",  
        "LILY OF THE VALLEY",  
        "SNAPDRAGON",  
        "HERBS",  
        "PETALS",  
        "NETTLE",  
        "HEATHER",  
        "GORSE",  
        "JUNIPER",  
        "RASPBERRY",  
        "LAVENDER",  
    ],  
    [  
        "OAK LEAVES",  
        "CATMINT",  
        "MAPLE SEED",  
        "LAUREL",  
        "BULB WHITE",  
        "BULB YELLOW",  
        "BULB ORANGE",  
        "BULB PINK",  
        "BULB BLUE",  
        "CLOVER",  
        "DAISY",  
        "ARROWHEAD",
```

```
    ],  
    ]
```

Don't actually do this for medcatherb, follow the medcatherb_data section below

If you placed your accessory(ies) on a separate row:

```
medcatherbs_data = [  
    [  
        "MAPLE LEAF",  
        "HOLLY",  
        "BLUE BERRIES",  
        "FORGET ME NOTS",  
        "RYE STALK",  
        "CATTAIL",  
        "POPPY",  
        "ORANGE POPPY",  
        "CYAN POPPY",  
        "WHITE POPPY",  
        "PINK POPPY",  
    ],  
    [  
        "BLUEBELLS",  
        "LILY OF THE VALLEY",  
        "SNAPDRAGON",  
        "HERBS",  
        "PETALS",  
        "NETTLE",  
        "HEATHER",  
        "GORSE",  
        "JUNIPER",  
        "RASPBERRY",  
        "LAVENDER",  
    ],  
    [  
        "OAK LEAVES",  
        "CATMINT",  
        "MAPLE SEED",  
        "LAUREL",  
        "BULB WHITE",
```

```

        "BULB YELLOW",
        "BULB ORANGE",
        "BULB PINK",
        "BULB BLUE",
        "CLOVER",
        "DAISY",
    ],
    [
        "ARROWHEAD",
    ],
]

```

Follow the medcatherb_data section below if you're editing this in particular

Always follow what you did to the PNG! Make sure your accessory names are in order corresponding to what you added to the PNG, plus make sure you're being mindful about their placement.

Accessory names should always be UPPERCASE

medcatherbs_data

Obviously this file is kinda complicated because it's separated into two different datas - medcatherbs_data and dryherbs_data. If you want to add your accessories to medcatherbs_data but it's behind the four accessories at the bottom of the file (aka dry herbs), then you'll need to reposition the accessories.

★ **NOTE:** If you intend to just keep the accessories with dry herbs, edit dryherbs_data instead

You can do this by adding a new row to the file and dragging dry herbs to the new row, then placing your additional accessories where the dry herbs originally were. This allows you to simply add your accessories to medcatherbs_data without mucking anything up

★ **NOTE:** Make sure when doing this, you change the enumerate(dryherbs_data) (col, 3) to be (col, 4)

Pelts.py

```
{scripts > cat folder}
```

Scroll through the file until you're at line 80, `plant_accessories`. This is where you categorize the accessories you added. Because I'm adding a tail accessory, I'll be placing mine in two different areas

```
plant_accessories = ["MAPLE LEAF", "HOLLY", "BLUE BERRIES", "FORGET ME
NOTS", "RYE STALK", "CATTAIL", "POPPY", "ORANGE POPPY", "CYAN POPPY",
"WHITE POPPY", "PINK POPPY",
                    "BLUEBELLS", "LILY OF THE VALLEY", "SNAPDRAGON",
"HERBS", "PETALS", "NETTLE", "HEATHER", "GORSE", "JUNIPER", "RASPBERRY",
"LAVENDER",
                    "OAK LEAVES", "CATMINT", "MAPLE SEED", "LAUREL",
"BULB WHITE", "BULB YELLOW", "BULB ORANGE", "BULB PINK", "BULB BLUE",
"CLOVER", "DAISY",
                    "CLOVER", "DAISY", "LILY OF THE VALLEY",
"HEATHER", "SNAPDRAGON", "GORSE", "BULB WHITE", "BULB YELLOW",
                    "DRY HERBS", "DRY CATMINT", "DRY NETTLES", "DRY
LAURELS", "ARROWHEAD"
]
```

Indentation does not matter - it's for readability

Only add your accessory here if the accessory is on the tail. This is important for later

```
tail_accessories = ["RED FEATHERS", "BLUE FEATHERS", "JAY FEATHERS",
"GULL FEATHERS", "SPARROW FEATHERS", "CLOVER", "DAISY", "ARROWHEAD"]
```

Cats.py & scar_events.py

IF your accessory is a tail accessory, follow these steps.

Navigate to `cats.py` (scripts folder > cats folder), scroll until line 2043, `# remove accessories if need be`

Here is the code that makes tail accessories invisible if a cat has a tail limb scar, such as no tail and half tail.

```
# remove accessories if need be
if "NOTAIL" in self.pelt.scars and self.pelt.accessory in [
```

```

        "RED FEATHERS",
        "BLUE FEATHERS",
        "JAY FEATHERS",
        "GULL FEATHERS",
        "SPARROW FEATHERS",
        "CLOVER",
        "DAISY",
        "ARROWHEAD",
    ]:
        self.pelt.accessory = None
    if "HALFTAIL" in self.pelt.scars and self.pelt.accessory in [
        "RED FEATHERS",
        "BLUE FEATHERS",
        "JAY FEATHERS",
        "GULL FEATHERS",
        "SPARROW FEATHERS",
        "CLOVER",
        "DAISY",
        "ARROWHEAD",
    ]:

```

You want to do this, otherwise your tail accessory will be flying off kitties with no tails.

Theoretically if you want to, you can also add code for other no limb scars, like NOPAW, if your accessory relies on the area the no limb scar removes.

You'll also have to edit scar_events.py (scripts > events_module > scar_events) if you want to make sure tail accessories can't be generated for no tail kitties either.

Scroll until about line 153, if specialty in ["NOTAIL", "HALFTAIL"]. Add your tail accessories here.

```

specialty = random.choice(scar_pool)
if specialty in ["NOTAIL", "HALFTAIL"]:

```

```
        if cat.pelt.accessory in ["RED FEATHERS", "BLUE FEATHERS",  
"JAY FEATHERS", "GULL FEATHERS", "SPARROW FEATHERS", "CLOVER", "DAISY",  
"ARROWHEAD"]:  
            cat.pelt.accessory = None
```

acc_display.json

(resources > dicts > acc_display.json)

This file holds all the text you see when the accessory is mentioned in an event or shown on a cat's profile.

Scroll all the way down to the end of the file, and duplicate the code of the last text code. The "id" ("ARROWHEAD") should be exact to how the accessory is written in the .py files.

```
"INFO": [  
    "default should be what you'd like to display on the cat profile",  
    "plural and singular will be used on events"  
],
```

```
},  
"INDIGONYLON": {  
    "default": "indigo nylon collar",  
    "plural": "indigo nylon collars",  
    "singular": "indigo nylon collar"  
},  
"ARROWHEAD": {  
    "default": "arrowhead",  
    "plural": "arrowheads",  
    "singular": "arrowhead"  
}  
}
```

Then boom, you're done! Run the game to make sure everything is working smoothly.

: Adding new PNG

PNG

Very, very simple. You'll just add your accessory(ies) to a new png and add it to the sprites folder. Write down the name of the png and the order of the accessories in the png somewhere for future reference.

Coding

Once the png is present in the sprites folder and you have everything written

down (name of png, name of accessories in order and row), then we'll head to coding!

For this section, I am going to use the file name "toxicplants" and the accessories in this order: (row one) "ARROWHEAD", "YUCCA" (row two) "LILIES", "STOCK"

Sprites.py

First file we're going to edit is sprites.py. This can be found within the path: scripts > cats > sprites.py. Scroll down to about like 150.

The first thing you have to do when adding a new png into the game is to make sure the game knows it's there. We do that by editing "for x in []"

```
for x in [  
    "lineart",  
    "lineartdf",  
    "lineartdead",  
    "eyes",  
    "eyes2",  
    "skin",  
    "scars",  
    "missingscars",  
    "medcatherbs",  
    "wild",
```

```
"collars",
"bellcollars",
"bowcollars",
"nyloncollars",
"singlecolours",
"speckledcolours",
"tabbycolours",
"bengalcolours",
"marbledcolours",
"rosettecolours",
"smokecolours",
"tickedcolours",
"mackerelcolours",
"classiccolours",
"sokokecolours",
"agouticolours",
"singlestripecolours",
"maskedcolours",
"shadersnewwhite",
"lightingnew",
"whitepatches",
"tortiepatchesmasks",
"fademask",
"fadestarclean",
"fadeddarkforest",
"symbols",
"toxicplants",
]:
```

Now that we added the png, we can add our accessories. Scroll down until about line 555, #accessories.

The accessories data placement doesn't really matter, but for organization and future updating, I am going to scroll down until after nyoncolours_data and add my own section - # Honey's mod

It isn't necessary, but when updating your mod it can be significantly faster if you comment where your additions are so you can just search for them later.


```

# Honeys mod
toxicplants_data = [
    ["ARROWHEAD", "YUCCA"],
    ["LILIES", "STOCK"],
]

for row, toxicplants in enumerate(toxicplants_data):
    for col, toxic in enumerate(toxicplants):
        self.make_group("toxicplants", (col, row),
f"acc_toxic{toxic}")

```

The first part, `toxicplants_data`, is the data of the png. This is telling the game what accessories are within the file. Each grouping of brackets signifies a row

- ★ indentation doesn't matter here. It's for readability. I like it when it's horizontal instead of vertical, so it's really up to preference

The second part - I basically just copied another accessories code and replaced what was necessary (highlighted in blue)

- ★ `toxicplants` is the name of the png - should be in three areas
- ★ `toxicplants_data` is essentially the accessories placement data. Should only be in one area
- ★ `toxic` is the type of accessory. It is to separate it from the others. Keep the name relevant to your accessories. Should be in two areas
- ★ `"acc_toxic"` is optional - you'll only edit this if you're adding a new accessory category. Otherwise, please use the existing categories (`"acc_herbs"`, `"acc_wild"`, `"collars"`)
 - (refer to `"utility.py"` if you want to add a category)

Pelts.py

Now we add the individual accessory names to the `pelts.py`! (`scripts > cats`) Scroll until around line 80, `plant_accessories`

If adding to an existing category:

- ★ Make sure you're adding the accessories to a category that corresponds with the code you added to `sprites.py`!
- "acc_herbs" accessories goes to `plant_accessories`
 - "acc_wild" accessories goes to `wild_accessories`
 - "collars" accessories goes to `collars`
 - Any accessory that covers the tail goes in `tail_accessories` (in this case, the accessory would be put in two different places)

```
plant_accessories = ["MAPLE LEAF", "HOLLY", "BLUE BERRIES", "FORGET ME
NOTS", "RYE STALK", "CATTAIL", "POPPY", "ORANGE POPPY", "CYAN POPPY",
"WHITE POPPY", "PINK POPPY",
                    "BLUEBELLS", "LILY OF THE VALLEY", "SNAPDRAGON",
"HERBS", "PETALS", "NETTLE", "HEATHER", "GORSE", "JUNIPER", "RASPBERRY",
"LAVENDER",
                    "OAK LEAVES", "CATMINT", "MAPLE SEED", "LAUREL",
"BULB WHITE", "BULB YELLOW", "BULB ORANGE", "BULB PINK", "BULB BLUE",
"CLOVER", "DAISY",
                    "CLOVER", "DAISY", "LILY OF THE VALLEY",
"HEATHER", "SNAPDRAGON", "GORSE", "BULB WHITE", "BULB YELLOW",
                    "DRY HERBS", "DRY CATMINT", "DRY NETTLES", "DRY
LAURELS", "ARROWHEAD", "YUCCA", "LILIES", "STOCK"
]
```

If adding a new category:

Refer to the section "utility.py" when adding new categories

```
plant_accessories = ["MAPLE LEAF", "HOLLY", "BLUE BERRIES", "FORGET ME
NOTS", "RYE STALK", "CATTAIL", "POPPY", "ORANGE POPPY", "CYAN POPPY",
"WHITE POPPY", "PINK POPPY",
                    "BLUEBELLS", "LILY OF THE VALLEY", "SNAPDRAGON",
"HERBS", "PETALS", "NETTLE", "HEATHER", "GORSE", "JUNIPER", "RASPBERRY",
"LAVENDER",
                    "OAK LEAVES", "CATMINT", "MAPLE SEED", "LAUREL",
"BULB WHITE", "BULB YELLOW", "BULB ORANGE", "BULB PINK", "BULB BLUE",
"CLOVER", "DAISY",
                    "CLOVER", "DAISY", "LILY OF THE VALLEY",
"HEATHER", "SNAPDRAGON", "GORSE", "BULB WHITE", "BULB YELLOW",
```

```

        "DRY HERBS", "DRY CATMINT", "DRY NETTLES", "DRY
LAURELS"
    ]
    toxic_accessories = ["ARROWHEAD", "YUCCA", "LILIES", "STOCK"]
    wild_accessories = ["RED FEATHERS", "BLUE FEATHERS", "JAY FEATHERS",
"GULL FEATHERS", "SPARROW FEATHERS", "MOTH WINGS", "ROSY MOTH WINGS",
"MORPHO BUTTERFLY", "MONARCH BUTTERFLY", "CICADA WINGS", "BLACK CICADA"]

```

Then scroll down def init_accessories and add your category to the below code

```

    if acc_display_choice == 1:
        self.accessory = choice([
            choice(Pelt.plant_accessories),
            choice(Pelt.wild_accessories),
            choice(Pelt.toxic_accessories),
        ])
    else:
        self.accessory = None

```

Cats.py & scar_events.py

IF your accessory is a tail accessory, follow these steps.

Navigate to cats.py (scripts folder > cats folder), scroll until line 2043, # remove accessories if need be

Here is the code that makes tail accessories invisible if a cat has a tail limb scar, such as no tail and half tail.

```

# remove accessories if need be
if "NOTAIL" in self.pelt.scars and self.pelt.accessory in [
    "RED FEATHERS",
    "BLUE FEATHERS",
    "JAY FEATHERS",
    "GULL FEATHERS",
    "SPARROW FEATHERS",
    "CLOVER",
    "DAISY",

```

```

        "ARROWHEAD",
    ]:
        self.pelt.accessory = None
    if "HALFTAIL" in self.pelt.scars and self.pelt.accessory in [
        "RED FEATHERS",
        "BLUE FEATHERS",
        "JAY FEATHERS",
        "GULL FEATHERS",
        "SPARROW FEATHERS",
        "CLOVER",
        "DAISY",
        "ARROWHEAD",
    ]:

```

You want to do this, otherwise your tail accessory will be flying off kitties with no tails.

Theoretically if you want to, you can also add code for other no limb scars, like NOPAW, if your accessory relies on the area the no limb scar removes.

You'll also have to edit scar_events.py (scripts > events_module > scar_events) if you want to make sure tail accessories can't be generated for no tail kitties either.

Scroll until about line 153, if specialty in ["NOTAIL", "HALFTAIL"]. Add your tail accessories here.

```

        specialty = random.choice(scar_pool)
        if specialty in ["NOTAIL", "HALFTAIL"]:
            if cat.pelt.accessory in ["RED FEATHERS", "BLUE FEATHERS",
                "JAY FEATHERS", "GULL FEATHERS", "SPARROW FEATHERS", "CLOVER", "DAISY",
                "ARROWHEAD"]:
                cat.pelt.accessory = None

```

Utility.py

{scripts > utility.py}

If you added a new category, edit utility!

Okay so! IF you want to add a new category of accessories, you'll need to make sure to add it to the utility sprites pool.

Copy below:

```
        elif cat.pelt.accessory in cat.pelt.collars:
            new_sprite.blit(
                sprites.sprites["collars" + cat.pelt.accessory +
cat_sprite], (0, 0)
            )
```

And **only edit cat.pelt.collars and "collars"** to fit your new accessory category.

- ★ cat.pelt.collars is the category listed in pelts.py.
 - For me, this is toxic_accessories, so I'd just make it cat.pelt.collars -> cat.pelt.toxic_accessories
- ★ "collars" is the category listed in the accessories code within sprites.py
 - For me, this is "acc_toxic", so I'd just make it "collars" -> "acc_toxic"

```
# draw accessories
if not acc_hidden:
    if cat.pelt.accessory in cat.pelt.plant_accessories:
        new_sprite.blit(
            sprites.sprites["acc_herbs" + cat.pelt.accessory +
cat_sprite],
            (0, 0),
        )
    elif cat.pelt.accessory in cat.pelt.wild_accessories:
        new_sprite.blit(
            sprites.sprites["acc_wild" + cat.pelt.accessory +
cat_sprite],
            (0, 0),
        )
    elif cat.pelt.accessory in cat.pelt.collars:
        new_sprite.blit(
```

```

        sprites.sprites["collars" + cat.pelt.accessory +
cat_sprite], (0, 0)
    )
    elif cat.pelt.accessory in cat.pelt.toxic_accessories:
        new_sprite.blit(
            sprites.sprites["acc_toxic" + cat.pelt.accessory +
cat_sprite], (0, 0)
        )

```

You can also go one step further and make a special accessory tag for your new category for accessory gain events, but I won't go over that since this is for art additions.

★ Here's the files you might have to edit:

- handle_short_events.py: def handle_accessories
- test_handle_short_events.py: (???)

acc_display.json

(resources > dicts > acc_display.json)

This file holds all the text you see when the accessory is mentioned in an event or shown on a cat's profile.

Scroll all the way down to the end of the file, and duplicate the code of the last text code. The "id" ("ARROWHEAD") should be exact to how the accessory is written in the .py files.

```

"INFO": [
    "default should be what you'd like to display on the cat profile",
    "plural and singular will be used on events"
],

```

```

},
"INDIGONYLON": {
    "default": "indigo nylon collar",
    "plural": "indigo nylon collars",
    "singular": "indigo nylon collar"
},

```

```

"ARROWHEAD": {
    "default": "arrowhead",

```

```
"plural": "arrowheads",  
"singular": "arrowhead"  
}  
}
```

Then boom, you're done! Run the game to make sure everything is working smoothly.

Adding to White Patches

Be aware you need the source (raw code) version of clangen to add additional options

There are a whopping 130 white patches (2,730 *individual* sprites) currently in ClanGen, and there will likely be more official additions in the future.

"What's a white patch?" Basically, a lot of cats have patches of white on their pelts in a lot of unique patterns. You might be familiar with the term "tuxedo" where a cat has white on their chest. That's a white patch!

The white patches PNG also holds the games vitiligo and colorpoint sprites. If you wish to add to that pool of sprites, you'll edit the whitepatches png!

PNG

Making white patches is fairly easy - you're just drawing a pattern in pure white on a png! Use the layout files I provide (in the google drive link) in "layout" for your convenience.

- ★ Make sure when you're done, you're deleting the background checker box pattern and lineart layer. Otherwise your sprites will look a little weird. Just a tad.

NOTE: If you want to do it ClanGen's way, here's the [official documentation](#)! It'll explain briefly how to draw the white patches in their style

Coding

Once you're happy with your white patches, replace and/or save the PNG. Make sure to write down somewhere the order of your additions, and if your additions are vitiligo/colorpoint. This is crucial later!

For my example, I'll be using "TAZ"!

Sprites.py

```
{scripts > cats}
```

Scroll down until about 216 - #define white patches - this is the white patches list.

White patches, vitiligo, and color points all go into this list.

Vitiligo and color points are categories in pelts.py, which I'll go over later~

Here's how "TAZ" would be added to white_patches

```
],
[
    "BULLSEYE",
    "FINN",
    "DIGIT",
    "KROPKA",
    "FCTWO",
    "FCONE",
    "MIA",
    "SCAR",
    "BUSTER",
    "SMOKEY",
    "HAWKBLAZE",
    "CAKE",
    "ROSINA",
    "PRINCESS",
],
["LOCKET", "BLAZEMASK", "TEARS", "DOUGIE", "TAZ"],
]
```


Adding TAZ to the last bracket category in white patches

```
[
    "BULLSEYE",
    "FINN",
    "DIGIT",
    "KROPKA",
    "FCTWO",
    "FCONE",
    "MIA",
    "SCAR",
    "BUSTER",
    "SMOKEY",
    "HAWKBLAZE",
    "CAKE",
    "ROSINA",
    "PRINCESS",
],
["LOCKET", "BLAZEMASK", "TEARS", "DOUGIE"],
["TAZ"],
]
```

*Adding TAZ to a new bracket. Only do this if you added a new row to the
png*

pelts.py

```
{scripts > cats}
```

Now, we're going to categorize your additions by how much they cover the pelt. There are 4 categories for white patches: `little_white`, `mid_white`, `high_white`, and `mostly_white`. **Vitiligo and color patches don't go in these.**

The way you interpret these categories is up to you. It's essentially just barely some white to covering the majority of the pelt.

```
mid_white = ['TUXEDO', 'FANCY', 'UNDERS', 'DAMIEN', 'SKUNK',
'MITAIN', 'SQUEAKS', 'STAR', 'WINGS',
```

```

        'DIVA', 'SAVANNAH', 'FADESPOTS', 'BEARD', 'DAPPLEPAW',
'TOPCOVER', 'WOODPECKER', 'MISS', 'BOWTIE',
        'VEST',
        'FADEBELLY', 'DIGIT', 'FCTWO', 'FCONE', 'MIA', 'ROSINA',
'PRINCESS', 'DOUGIE', 'TAZ']

```

Example adding TAZ to mid_white

Vit and Colorpoints

Now that you added your additions to white_patches above, we're going to now add any white patches meant to be vitiligo or colorpoint to the categories in pelts.py.

Pretending "TAZ" is both of those, this is what the addition would look like:

```

point_markings = ['COLOURPOINT', 'RAGDOLL', 'SEPIAPOINT', 'MINKPOINT',
'SEALPOINT', 'TAZ']
vit = ['VITILIGO', 'VITILIGOTWO', 'MOON', 'PHANTOM', 'KARPATI',
'POWDER', 'BLEACHED', 'SMOKEY', 'TAZ']

```

Adding to Tortie Pattern

Be aware you need the source (raw code) version of clangen to add additional options

There is some confusion surrounding tortie patterns so I'll explain the gist of it here: Tortie patterns is a patch (similar to white patches) that make a cat a tortie upon generating.

It is drawn as white, then the game uses the pattern to "overlay" a second color and pelt type (if wild tortie) on the pelt. This allows the cat to have the bicolor look.

Tortie patterns have 43 options in the spritesheet. This is 86 individual sprites.

PNG

Making tortie patches is fairly easy - you're just drawing a pattern in pure white on a png! Use the layout files I provide (in the google drive link) in "layout" for your convenience.

- ★ Make sure when you're done, you're deleting the background checker box pattern and lineart layer. Otherwise your sprites will look a little weird. Just a tad.

NOTE: If you want to do it ClanGen's way, here's the [official documentation](#)! It'll explain briefly how to draw the white patches in their style, which can be used for tortie patches

Coding

Once you're happy with your torie patches, replace and/or save the PNG. Make sure to write down somewhere the order of your additions. This is crucial later!

For my example, I'll be using "TAZ"

sprites.py

{scripts > cats} At about 398 is where you'll edit the file - define tortiepatchesmasks

```
[
    "ORIOLE",
    "ROBIN",
    "BRINDLE",
    "PAIGE",
    "ROSETAIL",
    "SAFI",
    "DAPPLENIGHT",
    "BLANKET",
    "BELOVED",
    "BODY",
],
["SHILOH", "FRECKLED", "HEARTBEAT", "TAZ"],
]
```

If you make a new row in the png, you'd make a new bracket section like below:

```
[
    "ORIOLE",
    "ROBIN",
    "BRINDLE",
    "PAIGE",
    "ROSETAIL",
    "SAFI",
    "DAPPLENIGHT",
    "BLANKET",
    "BELOVED",
    "BODY",
],
["SHILOH", "FRECKLED", "HEARTBEAT"],
["TAZ"],
]
```

pelts.py

{scripts > cats} Now we add your additions to "tortiepatterns" in the pelts.py file. Like below:

```
tortiepatterns = ['ONE', 'TWO', 'THREE', 'FOUR', 'REDTAIL', 'DELILAH',
'MINIMALONE', 'MINIMALTWO', 'MINIMALTHREE',
                  'MINIMALFOUR', 'HALF',
                  'OREO', 'SWOOP', 'MOTTLED', 'SIDEMASK', 'EYEDOT',
'BANDANA', 'PACMAN', 'STREAMSTRIKE', 'ORIOLE',
                  'CHIMERA', 'DAUB', 'EMBER', 'BLANKET',
                  'ROBIN', 'BRINDLE', 'PAIGE', 'ROSETAIL', 'SAFI',
'SMUDGED', 'DAPPLENIGHT', 'STREAK', 'MASK',
                  'CHEST', 'ARMTAIL', 'SMOKE', 'GRUMPYFACE',
                  'BRIE', 'BELOVED', 'BODY', 'SHILOH', 'FRECKLED',
'HEARTBEAT', 'TAZ']
```

And you're done!

Adding to Skins

Be aware you need the source (raw code) version of clangen to add additional options

Skins are 10 pixels max, and very easy to do unless you decide to be more complex with how they're created.

Because of the simplicity of the file, I don't think I need to explain how to create new skins. Please check out "layout" for the png files I recommend using while making new options!

Expanding Canvases

You will need to expand the canvas size, as the existing skins already take all the space within the file. Here's how I do it with

FireAlpaca:

- Navigate to the top and select "edit"
- Select "canvas size..." (or ctrl + alt + c)
- Specify top center since we're extending height
- Add 350 x how many new rows you need

If your program doesn't work the way mine does, please consider searching up "how to expand canvas side in [art program]" in your browser

Coding

Once your skins are done and you're happy with how they look, replace the files in the sprites folder with your modified one. Or just save if you're editing the files directly.

Make sure to delete the lineart and background layer when you're done! Otherwise it might look a little strange in game. Just a tad.

sprites.py

```
# Define skin colors
```

pelts.py

skin_sprites

Adding to Scars

Be aware you need the source (raw code) version of clangen to add additional options

This file is a doozy. Scars have

sprites.py

Define scars

pelts.py

scars from other cats, other animals

Adding to tints

Tints don't require the source version, unless it involves adding additional options elsewhere. The tints files are in your sprites > dicts folder.

Adding Symbols

Be aware you need the source (raw code) version of clangen to add additional options

Symbols came with the leader's den feature in release 0.12.0. The symbols are reliant on prefixes that can be generated as clan names, aka the names.png file.

Adding Camps

Be aware you need the source (raw code) version of clangen to add additional options

Yup, there sure is! Here's a brief run down

- * 8 pngs of your camp is required. Light mode and dark mode for each season. It's not required that they're different visually, but you need 8 separate pngs

- * Those pngs need to be named in a specific way: season_camp#_mode -- you'll need to add them to resources > images > camp_bg > desired biome (can also find examples in there as well)

- * Find the placements.json in the resources folder. If this is your first time doing UI placements, I recommend deliberately creating your camp with the same layout of an existing one so you can just copy paste the placements of an existing camp for your camp. If you do copy paste, simply change the UI's "ID" with "Biomecamp#", like "Beachcamp5"

- * From there, you will need to edit the makeclanscreen.py in your script > screens folder. This is where you add the button to the selection screen so your camp is available. Search ``def refresh\_selected\_camp``, scroll until you see the biome you want to add to (``self.biome\_selected == "desired biome"``) and add onto that section of code by simply copy and pasting what already exists, renaming the tab, and changing the tab number to 5

```
tab_rect = ui_scale(pygame.Rect((0, 0), (80, 30)))
tab_rect.topright = ui_scale_offset((5, 5))
self.tabs["tab5"] = UISurfaceImageButton(
    tab_rect,
    "Slope",
    get_button_dict(ButtonStyles.VERTICAL_TAB, (80, 30)),
    object_id="@buttonstyles_vertical_tab",
    manager=MANAGER,
    anchors={
        "right": "right",
        "right_target": self.elements["art_frame"],
        "top_target": self.tabs["tab4"],
    },
)
```

- * After that's done, you'll likely have to add a new self.tab line for tab5 (search ``self.tabs["tab4"]``). It'll look something like:

```
        self.tabs["tab5"].disable() if self.selected_camp_tab == 5 else
self.tabs[
    "tab5"
].enable()
```

That should be everything you need to do. Hopefully I explained this well enough

Adding Patrol Art

Very simple! Make art, add to resources > images > patrol art. Find the patrol you want to add the art to in the game files, add patrol art name (without .png) in the art area.